

Nishan-e-Zindagi: A Distributed Framework for Search-and-Rescue in Disaster Relief



A Final Year Senior Design Project Report

Submitted by:

Ahmed Musharaf (2022067)

Amaan Zahir Sadiq (2022902)

Saaim Ali Khan (2022519)

Advisor:

Hafiz Syed Muhammad Muslim

Co-Advisor:

Dr. Masroor Hussain

Faculty of Computer Science and Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and
Technology

April 2026

Certificate of Approval

It is certified that the work presented in this report was performed by **Ahmed Musharaf, Amaan Zahir Sadiq, and Saaim Ali Khan** under the supervision of **Hafiz Syed Muhammad Muslim** and co-supervision of **Dr. Masroor Hussain**. The work is adequate and lies within the scope of the BS degree in Computer Engineering at Ghulam Ishaq Khan Institute of Engineering Sciences and Technology.

Hafiz Syed Muhammad Muslim

(Advisor)

Dr. Masroor Hussain

(Co-Advisor)

Prof. Dr. Qadeer Ul Hassan

(Dean)

SDGs Inclusion

Nishan-e-Zindagi contributes to three of the United Nations Sustainable Development Goals (SDGs). We make contributions to all of them practically, as opposed to aspirationally. The Nishan-e-Zindagi system is made up of hardware and software elements that are specifically designed to take action to contribute to each SDG by implementing a feature that would normally be difficult to implement.

SDG 9: Industry, Innovation, and Infrastructure

This is an example of a way to provide resilient, low cost sensing infrastructure for disaster relief/ response. Since it doesn't need cell phone or internet connectivity it can function in locations where those services aren't available. This provides a model for how critical infrastructure can be constructed using locally sourced materials at low cost. The documentation about the hardware and software parts of this system will support local manufacturing/maintenance; a factor important to developing countries that have difficulty importing sophisticated SAR equipment.

SDG 11: Sustainable Cities and Communities

Urban cities in Pakistan experience high levels of disaster risk due to earthquake related hazards, flood-related hazards, and failure of buildings. For example, cities such as Karachi and Muzaffarabad sit in seismically active regions with building stocks that are poorly suited for survival during major events. Deploying an affordable, post-disaster sensor network for search and rescue purposes will increase the resiliency of urban communities. Locating survivors sooner using sensors that first responders can affordably purchase and maintain represents a direct contribution to urban resiliency.

SDG 15: Life on Land

In addition to urban applications, the system is designed to operate in obstructed terrain (such as hills, forests, etc.) typical of northern Pakistan. Response to landslides, flash flooding, and avalanche-type disasters in these types of environment is frequently hampered by limited communications and accessibility. A mesh-networked sensor system operating independently of existing infrastructure will extend response capabilities into these challenging environments supporting both human life and, indirectly, the ecosystems associated with disaster response activities (e.g., via fewer helicopter and/or heavy equipment based responses).

Hafiz Syed Muhammad Muslim

(Advisor)

Dr. Masroor Hussain

(Co-Advisor)

Abstract

When a structure collapses, time begins. The majority of this 72-hour window will be used to simply determine how to proceed. This is because current (and future) technologies designed for search and rescue operations have their own limitations. Each technology does something better than another technology but also has its weaknesses. Thermal imaging requires direct line-of-sight to the target area. Canine units will eventually fatigue. Hand-clearance of debris is both time consuming and dangerous to the rescue personnel involved.

In contrast to these single tool approaches Nishan-e-Zindagi, it uses a "sensor-node" based approach. Sensor-nodes are dispersed throughout the site and communicate with each other. Each sensor-node includes CO₂ sensors, temperature and humidity sensors and a GPS module connected to a Raspberry Pi Pico microcomputer and SX1262 LoRa wireless communication module using Meshtastic firmware. It is logical to assume that elevated levels of CO₂ in confined spaces are indicative of human presence. Therefore when multiple sensor-nodes detect elevated CO₂ levels in proximity to each other; then those areas warrant further investigation.

Sensor-nodes can be networked together as a mesh. Data packets can jump from sensor-node to sensor-node until they reach a designated gateway or "hub". From there data packets are transmitted via WebSockets to a web-based dashboard. The dashboard displays sensor-nodes plotted onto a map. The color of each sensor-node corresponds to its operational status. Additionally the dashboard provides real-time tracking of sensor-node battery life and received signal strength. Lastly, the dashboard plots real-time CO₂ level and humidity

readings from each sensor-node. An intermediary Python program resides between the gateway/hub and the dashboard.

ACKNOWLEDGEMENTS

This project would have been impossible without Sir Hafiz Syed Muhammad Muslim who is our advisor. He challenged us to go beyond simple answers by asking some of the most difficult questions we were trying to avoid. He also found the time to meet with us whenever we showed up at his office with only partially functional hardware.

Dr. Masroor Hussain, our other advisor, provided us with the framework to take what was essentially just a collection of sensors and make it a complete system. His input regarding the Anomaly Detection Logic for example likely saved us months.

GIKI's Faculty of Computer Science & Engineering (FCSE) allowed us to experiment and test. Because we had access to labs outside of regular working hours we were able to perform "soak" tests overnight; and because the lab staff was patient with all the soldering iron use that was definitely not scheduled during the week, we were able to get everything done quickly.

We would like to express our gratitude to our family members for their support as well as their patience through long nights, missing meals, and occasionally receiving lithium battery shipments to our home.

The *Meshtastic* Open Source Community is worth mentioning. Their maintained firmware supported the network layer of our project and afforded us an opportunity to complete this project within our budgetary constraints and time frame. We built upon their efforts throughout.

Ahmed Musharaf

Amaan Zahir Sadiq

Saaim Ali Khan

TABLE OF CONTENTS

CHAPTER I.....	1
Introduction.....	1
1.1 Context and Background.....	1
1.2 Motivation.....	2
1.3 Project Objectives	3
1.4 Main Contributions	6
1.4.1 A low-cost sensor node design	6
1.4.2 Python gateway implementation.....	6
1.4.3 Web dashboard.....	6
1.4.4 Raw field data collection	6
1.4.5 Failure modes and lessons learned.....	7
1.5 Report Organization.....	7
CHAPTER II.....	9
Literature Survey	9
2.1 Search and Rescue Technology	9
2.2 LoRa and Mesh Networking.....	11
2.3 CO2 and Humidity as Detection Signals	13
2.4 Gaps in Existing Work.....	15
2.5 Comparative Summary	16
2.6 Research Gap Statement	16

CHAPTER III	16
Design: System Requirements and Specifications.....	19
3.1 Stakeholders and User Classes.....	19
3.2 Functional Requirements	21
3.3 Non-Functional Requirements	24
3.3.1 On performance	24
3.3.2 On power.....	25
3.3.3 On range.....	25
3.3.4 On capacity	25
3.3.5 On reliability	25
3.3.6 On security	25
3.4 Design Constraints	26
3.4.1 Budget	26
3.4.2 Regulatory.....	26
3.4.3 Power	27
3.4.4 Bandwidth.....	27
3.4.5 Connectivity	27
3.4.6 Development time.....	27
3.5 External Interfaces	28
3.6 Primary Use Cases	29
CHAPTER IV	31
Proposed Solution: Methodology and Implementation	31

4.1 High-Level Architecture	31
4.2 Hardware Design	32
4.3 Node Firmware	36
4.4 Mesh Network Configuration	40
4.5 Gateway Service	42
4.6 Web Dashboard.....	46
4.7 Anomaly Detection Logic.....	49
4.8 Reporting Module	52
4.9 Implementation Notes and Lessons Learned	54
CHAPTER V	57
Results and Discussion	57
5.1 Bench Tests.....	57
5.2 Network Range and Reliability.....	59
5.3 Four Condition Simulation	61
5.4 Discussion	64
5.5 Comparative Evaluation.....	66
CHAPTER VI	68
Conclusion and Future Work	68
6.1 Conclusion	68
6.2 Future Work	69
6.2.1 Multi-node simultaneous testing.....	69
6.2.2 Power management.....	70

6.2.3 Acoustic detection.....	70
6.2.4 Format field validation.....	70
6.2.5 LoRa firmware customisation.....	71
6.2.6 Dashboard role hierarchy	71
6.3 Reflections on the Project	71
6.4 Limitations	72
6.4.1 Single-node proof of concept.....	72
6.4.2 Obstruction material.....	73
6.4.3 Battery life	73
6.4.4 GPS in deep burial	74
6.4.5 Dashboard load	74
6.4.6 Single administrator dependency	75
GLOSSARY	76
REFERENCES	79
APPENDIX A.....	81
Hardware Bill of Materials and Assembly.....	81
A.1 Complete Bill of Materials.....	81
A.2 Wiring	82
A.3 Assembly Steps	83
A.4 Enclosure.....	84
APPENDIX B	85
Raw Test Data Summaries.....	85

B.1 Open-Field Range Test.....	85
B.2 Building Penetration Test.....	85
B.3 Battery Discharge Profile	86

LIST OF FIGURES

Number	Caption.....	Page
1.1	Deployment concept diagram showing sensor nodes scattered across a disaster site.....	5
3.1	Use case diagram of Nishan-e-Zindagi System	21
4.1	System architecture — three-tier breakdown (sensory, hardware interface, storage/display).....	32
4.2	Node block diagram showing hardware interconnections	34
4.3	Firmware main loop flowchart	39
4.4	LoRa mesh topology example with four nodes and one gateway.....	40
4.5	Gateway service architecture	43
4.6	Sequence diagram tracing a single sensor	45
4.7-a	Admin map view with active nodes and alert panel	46
4.7-b	Rescuer map view with active nodes and alert panel.....	47
4.7-c	Public view with active nodes and alert panel	47
4.8	Anomaly detection decision flow	51
4.9	Aggregate report generation by the system	54
5.1	CO2 concentration over time during indoor breathing test.....	58
5.2	Packet delivery ratio vs distance in open-field conditions.....	60
5.3	Deployment map for simulated rubble pile test.....	63
5.4	Battery discharge curves across four test nodes over 48 hours.....	65

LIST OF TABLES

Number	Caption.....	Page
2.1	Comparison of existing SAR detection systems against Nishan-e-Zindagi	16
3.1	Functional requirements summary with priority levels	23
3.2	Non-functional requirements and measured values	26
4.1	Bill of materials for a single sensor node.....	35
4.2	Meshtastic channel configuration parameters.....	42
5.1	CO2 accuracy test results for a single node tested eight times	59
5.2	Packet delivery results for simulated rubble pile	61
A.1	Complete Bill of Materials	81
A.2	Wiring	82
B.1	Open-Field Range Test	85
B.2	Building Penetration Test	85
B.3	Battery Discharge Profile.....	86

CHAPTER I

Introduction

1.1 Context and Background

The first hour or two after an emergency happens during search and rescue in the aftermath of an earthquake, a collapse of buildings or a landslide is basically about getting as many people into the field as possible.

As soon as responders arrive at an area they know there will be people trapped somewhere in the affected area but they do not have any further information.

The responders then start to assess the damage -- identifying areas that may still contain viable voids within collapsed structures (i.e., empty space) and areas that are no longer capable of being searched for survivors due to extreme compression and therefore cannot be reached. They also determine how best to utilize their limited resources.

In Pakistan there are numerous active fault lines. The Kashmir Earthquake that occurred in 2005 resulted in over 87,000 deaths. More recently, in 2013 a major earthquake occurred in Awaran and while it was significantly smaller than the Kashmir event it still demonstrated the potential of earthquakes to cause widespread destruction. Additionally, the region experiences frequent flash flooding. While these events are certainly real and pose a legitimate risk of disaster, they are typically responded to through a combination of military personnel (typically from one of the three branches), members of the Pakistani Army's Engineer Corps, civilian volunteers and foreign disaster relief teams. The response efforts include a variety of tools and equipment including expensive thermal imaging equipment and specialized seismic monitoring

systems borrowed from other organizations that have greater financial resources available to them. However, the majority of tools used for search and rescue efforts are relatively basic such as hand held thermal imaging cameras, handheld seismic monitoring equipment, digging tools (shovels, etc.) pry bars and trained canines.

1.2 Motivation

The original concept was generated by an easier query; What do you do when a disaster zone is so hazardous for emergency personnel that they cannot remain on-site while there are likely survivors under debris? While thermal imaging requires an individual to point the camera and dogs have to be moved in shifts with their handlers, even seismically-listening geophones need technicians at each location.

Unlike these types of units which require some form of human interaction, a sensor unit can be deployed and simply left unattended. When the unit remains intact, it will continue to transmit data. When the unit is damaged beyond repair, it may be replaced at a relatively low cost. Furthermore, as long as the conditions within the unit's environment change, this change itself will carry useful data. An elevated stationary reading could indicate trapped air pockets from before the building collapsed. Conversely, readings that increase in intensity over time (for example, rising CO₂ levels in a pocket of rubble over twenty minutes), would provide more valuable data.

In addition, we desired to develop a product that didn't rely upon cellular networks. Cellular network availability typically ceases in disaster zones quickly due to either damage to tower structures or excessive usage on remaining operational towers. We were able to create a local mesh network using LoRa and Meshtastic protocols allowing our nodes to communicate with

each other. One of the nodes happened to connect to a laptop serving as the “dashboard.” In cases where the internet connection returns, we can resync. In cases where it doesn’t return, operations continue normally.

1.3 Project Objectives

Our primary goals when we started this project were:

Firstly, we had to design a low cost (less than \$40) "sensor node" that could be built in batches of less than 10 units, costing approximately \$450 per unit. This was not an unrealistic goal, but rather a constraint since if our system would cost more than a thermal imaging camera per unit, it wouldn't make sense for it to exist. We exceeded this cost by a wide margin as the BOM for the individual nodes is approximating \$32 (PKR 9000).

Secondly, we needed the mesh network to function over large areas such as those present in typical disaster scenarios. For example, a fallen apartment building may stretch over 50 meters and a landslide affected area could extend several kilometers. We wanted to achieve a minimum line of sight distance of 1 kilometer with relatively good packet loss, and to penetrate at least two or three layers of reinforced concrete. We achieved both using the Meshtastic software with the SX1262 module and SF=11.

Thirdly, we needed to develop a fully functional user interface that a non technical coordinator could utilize. Not a custom made UI developed specifically for the research effort, but a usable UI that someone managing an active search/rescue operation could launch on their laptop and begin to use immediately.

Fourthly, we required all components to have been thoroughly tested from start to finish. As such, we selected one node and subjected it to 4 testing

environments: 1. One person, 2. Three people, 3. A pile of clothing, and 4. A combination of a pile of clothing and blanket. Each environment tested the response of the sensor to varying degrees of human presence and insulation. Additionally, we created a functioning video demonstration that showed the node operating independently, captured base-line readings and then had two students blow directly onto the sensor to simulate a high concentration of carbon dioxide - capturing the rapid increase in reading and subsequent dashboard alert as a representation of the entire pipeline.

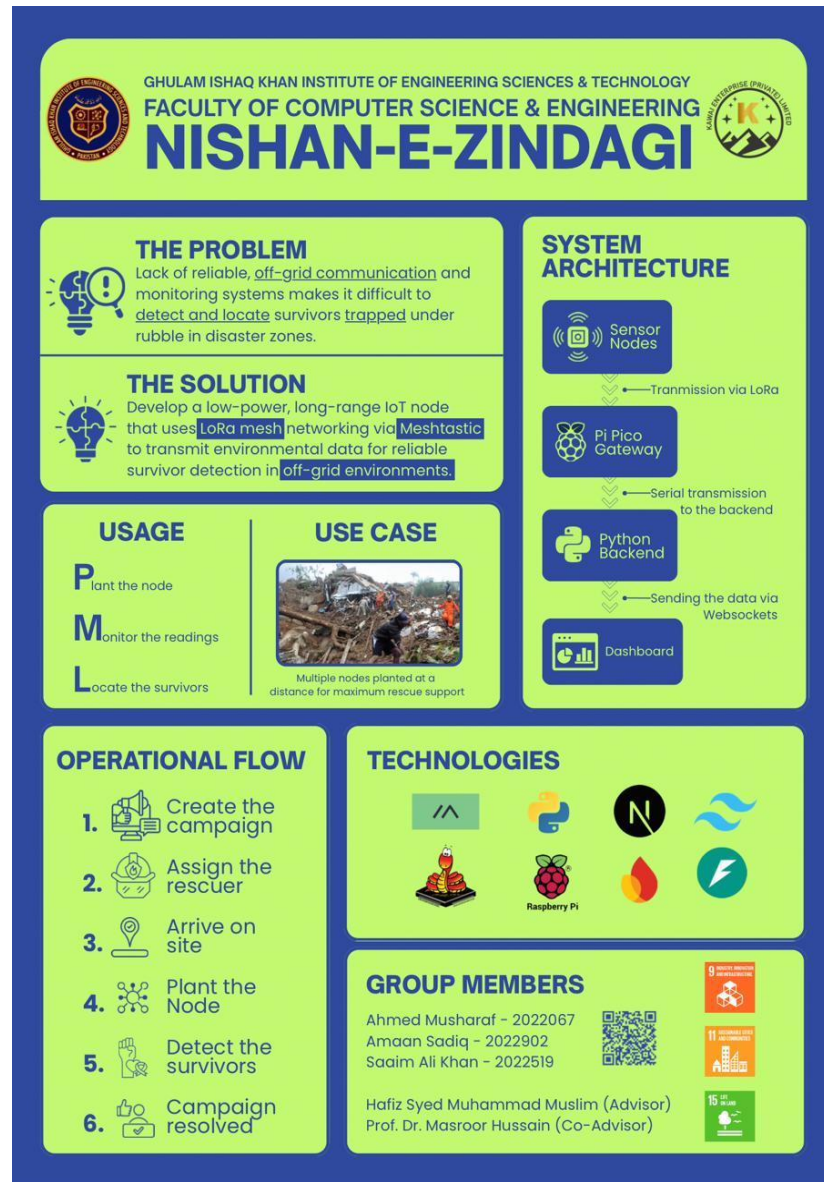


Fig. 1.1. Project overview poster showing the deployment concept, system architecture, and technology stack.

1.4 Main Contributions

We believe our project has made some valuable contributions:

1.4.1 A Low-Cost Sensor Node Design: A full documentation of the sensor node with firmware, bill of materials (BOM), enclosure designs and assembly instructions. With modest electronics knowledge, someone could build their own node in a weekend.

1.4.2 Python Gateway Implementation: An implementation of a light weight gateway using python. The gateway receives serial data from an active Meshtastic USB device and sends it to any connected browser client via WebSocket. The gateway separates the Meshtastic Protocol layer from the application layer.

1.4.3 Web Dashboard: Developed on top of Next.js. Features include live map views of all nodes, graphed telemetry information for each node, alerts based off of user defined rules, and daily reports based upon defined criteria. The web dashboard is intentionally kept very basic. We have tried to make it easy enough to quickly determine what is going on, where it is occurring, and how much confidence you should place in the detections being generated.

1.4.4 Raw Field Data Collection: Raw Field Data was also collected and analyzed from a single node in a variety of different test sessions. Battery life was tested along with packet delivery at varying ranges and under various obstructions and CO2 levels. All field data is included in chapter five of the document and also available in raw format in appendix B.

1.4.5 Some Failure Modes and Lessons Learned from Building the

Hardware That May Be Worth Knowing Beforehand: As stated above, we first designed the entire circuitry of our node on a breadboard before transitioning to a perfboard. While this process allowed us to catch many potential wiring or placement errors prior to placing the components onto the final board, there were still some issues that arose once we transitioned. One example of such an issue is that the NEO-6M GPS Module is relatively slow to obtain a "cold" fix when operating inside an enclosed space. For this reason, we had to seed the module with previously obtained cached coordinates so as to shorten the time required to initially lock onto a signal. Examples like these are ones that will typically never be mentioned within any datasheet.

1.5 Report Organization

The remainder of this document will be presented in the following order:

Chapter 2 explores existing works in this area. It includes a survey of technologies used for search and rescue in general; an overview of the use of sensors networks for disaster response; and explanations of the tools (LoRa, Meshtastic and the SCD41) that were selected.

Chapter 3 is the formal design section of the report. In this section we present our detailed system requirements and specifications. This includes descriptions of both the functional requirements of the system, its non-functional requirements, details regarding the user's interaction with the system, and details describing the system interfaces.

In Chapter 4 we describe how we implemented the components described in Chapter 3. This includes descriptions of how the hardware was designed and constructed; how the firmware for each component was developed; how the

gateway was implemented; how the dashboard was implemented; and how we determined when an anomaly had occurred. By far, this chapter has the longest description due to the amount of detail that needed to be included as well as the number of iterations that were performed during development.

Beginning in Chapter 5, we present results. We present results for individual component lab bench testing (individual components), field testing (of individual components), four-condition simulation where the sensor node was exposed to one person, three persons, a stack of cloth, and a combination of a stack of cloth and a blanket; as well as test session data from a single node. Results are provided through numeric quantities and we are transparent regarding those which were less than expected.

Chapter 6 discusses what it all adds up to and what we would be able to accomplish if we had another six months. That list has a lot on it.

Following Chapter 6 will be References, Glossary of Terms, and Appendices. The contents of Appendices A and B are as follows:

Appendix A is an overview of the Hardware Bill of Materials (BOM) and Assembly. Appendix B is a compilation of Raw Test Data.

CHAPTER II

Literature Survey

In this chapter we'll provide an overview of research in several fields that intersect with the areas of our Project: Search and Rescue Technologies; Low Power Wide Area Sensor Network Technology; Environmental Sensing of Human Presence. We will NOT attempt to present a comprehensive literature survey for each of the above (that would be enough for many dissertations). Instead, we'll discuss prior research in these fields that have impacted our Design Choices and identify the gaps that our implementation fills.

2.1 Search and Rescue Technology

Since the 1970s there has been active search and rescue research. Yet, technology used by first responders to perform their job has progressed at a slower rate than one would reasonably anticipate. Today's average urban search and rescue team utilizes some of the same methods and technologies that were utilized during the 1970s; although many of these new technological advancements were integrated into existing methods.

The most commonly employed detection method is thermal imaging. FLIR and Seek offer handheld thermal imagers capable of detecting a human temperature signature at ranges that allow for use by a typical urban search and rescue team. However, thermal imaging is limited to applications where the target area is visible to the thermal imager. When individuals are buried under a meter of concrete and unable to escape due to entrapment, they are essentially undetectable using thermal imaging; since concrete is a sufficient

insulation barrier and the heat signatures produced by humans are diminished and dissipated throughout the surrounding environment.

Acoustic detection is another commonly employed method. The Delsar LD3 places geophones and/or microphones over areas being searched and listens for sounds made by individuals within those areas. These devices produce data that is interpreted by an operator. While acoustic detection can detect sound made by individuals beneath layers of debris and may prove to be superior to thermal imaging when attempting to locate trapped persons, its ability to accurately interpret and relay information is significantly impaired when there is ambient noise present in the area. Ambient noise includes but is not limited to: the sounds produced by heavy machinery; the sounds generated by electrical generators; and the normal ambient noises associated with active disaster response operations. In addition, acoustic detection will only be successful when the individual trapped is conscious and able to generate audible noise.

Search teams utilizing canine units remain an essential resource in the search process. Canines trained to assist in searches can navigate through debris; recognize and follow human scent trails; and communicate findings to handlers. Practical constraints limit the effectiveness of canine search teams including but not limited to: the necessity for canine units to receive adequate rest periods before resuming duty; the inability for large-scale deployments to occur with canine teams; and the time required to train a qualified search dog (typically years).

Literature regarding sensor-based detection without reliance on line-of-sight visibility or direct operator input is less prevalent. Zhang et al. (2018) outlines a multi-modal solution utilizing CO₂ sensors, thermal imaging, and microphones as part of their design with primary concerns related to

operational costs and potential complexity associated with deploying such systems. Vellingiri et al. (2023) reports on a similarly designed low-cost CO₂ and humidity based system tied to a rescue robotics platform which enhances deployability while increasing overall system hardware and operating costs. Jadeja et al. (2024) goes farther, still as they implement sensors on snake-like robots coupled with edge-based deep learning models. This combination represents a highly interesting concept; however, it is likely impractical for wide scale deployment due to excessive cost and complexity considerations.

As we review all previously referenced studies, we notice that researchers generally choose to integrate additional functionalities into each respective device (i.e., more sensors, more intelligence, etc.), resulting in improved performance relative to answering questions posed about a singular aspect of the detection system while negatively impacting scalability. Conversely, our proposed methodology takes a different approach - each node is relatively simple compared to previous designs; multiple nodes are used; and intelligence is located on servers.

2.2 LoRa and Mesh Networking

LoRa is a proprietary physical-layer technology for long-range, low-power wireless communications, developed by Semtech, and commercially available since 2015. A unique feature of LoRa is its use of a chirp spread spectrum modulation. In this modulation scheme, trade-offs are made between the data-rate and the sensitivity/receiving distance. LoRa transmitters operating at 14 dBm can be received at greater than -135 dBm by compatible receivers, thus enabling link-budgets greater than 150 dB, and LOS ranges measured in kilometers.

In addition to the LoRa physical layer, several network layers were also created. The most commonly used is LoRaWAN, however, it uses a star topology. This star topology means that a network consists of a central "gateway" connected to a "network server". As such, LoRaWAN is intended for commercial IoT applications, where IoT endpoints will report data to a cloud service. While LoRaWAN performs well under normal operation conditions, the nature of a star topology makes it less than ideal for disaster response networks. When a single gateway goes offline, all of the network's users go offline.

Meshtastic approaches LoRa-based communication differently. Meshtastic is an open source application, running on LoRa based hardware. Meshtastic implements a mesh-protocol. Mesh protocols create multi-hop paths for packet relaying. Each node can communicate with any other node as long as they are within hearing of one another through their multi-hop path. Meshtastic provides features including packet-flooding, packet-re-transmit/acknowledgment at a layer above raw LoRa and below IP. Originally developed for off-the-grid communication needs (hiking/backpacking, etc.) - the characteristics that allow Meshtastic to provide viable communication solutions in these environments also enable it to provide viable solutions in post-disaster urban collapse situations.

Prior to selecting Meshtastic, we tested three different network stacks using our LoRa hardware: 1) raw LoRa with a custom protocol, 2) LoRaWAN with a gateway, and 3) Meshtastic. Using raw LoRa would have given us complete control over the communication layer, but implementing multi-hop routing from scratch requires significant time and effort we could not justify when the problem has already been solved by others. LoRaWAN was ruled out because it uses a star topology requiring fixed infrastructure, making it unsuitable for

ad-hoc deployment in disaster zones. Meshtastic provided working mesh networking with minimal configuration beyond getting the hardware running.

The primary challenges to large scale, dense LoRa network deployments identified by Rahman et al. (2022), includes; the maximum time it takes for data to be sent via a mesh (latency), and how long nodes can continuously send data before needing to rest (duty cycle). We found evidence of both issues during our testing. The test included sending a packet of data every 30 seconds. It took about 5 seconds after the data was sent until the data appeared on the dashboard. In addition we ran into the duty cycle problem while conducting our tests. When no JSON packets were received, we had to restart the node to get them again. Both issues still exist today for any new multi-node system built upon this technology.

2.3 CO2 and Humidity as Detection Signals

Carbon dioxide (CO₂) was selected as the first priority sensor in this study, and the decision was made based on the amount of CO₂ produced by a person during rest. On average, each person will produce approximately 250 ml (or 8.45 oz.) of carbon dioxide every minute through exhalation. This amount of gas will continue to increase the total CO₂ level of an enclosed area until there is no longer a source of additional CO₂. Today, the baseline CO₂ level outdoors is approximately 420 parts per million (ppm). In most cases where a living individual is placed into a relatively small enclosure or void, the levels of CO₂ will double within a short period of time after placement in the room, such as 20 – 40 minutes. The resulting increase in CO₂ concentrations may vary due to several factors including; the volume of the enclosed area (void), the flow of fresh air into the area (ventilation), and the rate at which the individual breathes (respiratory rate).

Use of CO₂ as a means of detecting human presence has been studied extensively in the field of Search and Rescue (SAR) since the early 1990's. These studies have used photoacoustic non-dispersive infrared (NDIR) sensors. However, these early studies were limited in accuracy, and larger in size, while requiring higher power input. More modern versions of NDIR sensors (such as the Sensirion SCD41) are two times smaller and lower cost than previous models. They also provide improved accuracy in their measurements of CO₂ levels (accurate to ± 50 ppm) across a measurement range that includes the values relevant to this study (400-5,000 ppm). Additionally, the SCD41 requires only low power input, and therefore can be powered directly from a microcontroller rail.

Humidity is an obvious companion to CO₂ when considering methods for detecting human presence in an enclosure. Each breath exhaled by humans is nearly saturated with water vapor. Therefore, if an individual is trapped in a closed environment and respires rapidly enough and/or long enough, they will increase the ambient relative humidity in that environment. Typically, an individual will create measurable changes in ambient relative humidity at a faster rate than they create measurable increases in ambient CO₂ levels. This is primarily due to the fact that ambient relative humidity is much farther away from 100% saturation than it is from zero percent saturation.

As previously mentioned, we rely upon data collected from the SCD41 for measuring CO₂ and humidity as well as temperature. Specifically for measuring humidity, we measure how fast the humidity reading is changing rather than its actual measured value. This is because the actual measured value of humidity is highly dependent on conditions outside of our control.

2.4 Gaps in Existing Work

The existing literature has left us with a deployable, distributed sensing system which can be made at low cost and therefore deployed on-site in large numbers. Much of what has been written about the subject, however, involves either a complex and expensive single instrument, or a small scale laboratory prototype. Very few publications suggested anything resembling an item a rescue coordinator would purchase and use. Nishan-e-Zindagi was developed to address this void -- It is a multi-node mesh sensing system, although for the purpose of this project, we have proven the concept of mesh sensing using a single node. Although the single-node version of Nishan-e-Zindagi represents a complete sensing and communications pipeline, the underlying architecture will support any number of nodes necessary for each unique deployment.

The distance (or gap) between research regarding sensor networks and commercially available field equipment -- That's where we focused this effort. We didn't develop any of the technology used in the development of Nishan-e-Zindagi. Instead, we took well established components and configured them together; then demonstrated that those components working together were successful.

2.5 Comparative Summary

Table 2.1. Comparison of existing SAR detection systems against Nishan-e-Zindagi.

System	Sensing Approach	Key Limitation	Our Response
Zhang et al. (2018)	CO ₂ + thermal + audio	High cost; needs line of sight	Drop thermal; use cheap distributed CO ₂ sensing instead
Vellingiri et al. (2023)	CO ₂ + humidity on rescue robot	Robot platform cost; limited rubble access	Static nodes placed manually; cheaper, more flexible
Jadeja et al. (2024)	Snake-robot sensors + edge DL	Complex hardware; heavy compute at edge	Centralized analysis; edge stays simple
Zafari et al. (2019)	IoT micro-location + GSM	GSM unreliable in disaster zones	LoRa mesh removes cellular dependency
FLIR K-series (commercial)	Thermal imaging	No penetration through concrete	CO ₂ works through rubble; complementary to thermal
Delsar LD3 (commercial)	Acoustic listening	Needs quiet site; victim must make noise	Passive sensing; survivor need not be conscious

2.6 Research Gap Statement

The four systems mentioned earlier have something else in common. Zhang et al., [1] and Vellingiri et al., [2], each used thermal imaging combined with CO₂ sensing. Both of these methods require line-of-sight from the sensors to the object being measured, which will not work when the area has been filled with smoke/dust as well as when objects are buried. Jadeja et al., [3], did get

good penetration results with their sensor-equipped snake robots, however the cost of implementing a robot in the middle of a collapsed structure is high. A robot that becomes stuck or is damaged is not going to help someone trying to find survivors. Zafari et al. [4], focused on providing reliable wireless communication for the nodes through GSM, which is typically one of the first services lost during disasters due to failure of cell towers.

All four systems have limitations to how they could be deployed in practice. None of them meet the requirements for a practical field-deployable SAR sensor that needs to include: (i) low cost per node, (ii) ability to communicate without relying upon pre-existing communications infrastructure, and (iii), a small and lightweight physical configuration so that a layman may deploy the sensor(s) in the rubble field. Zhang et al. [1] and Vellingiri et al. [2] have addressed the sense-making component, but are dependent upon expensive thermal imaging equipment. Jadeja et al. [3] have solved the burial issue by utilizing robots, but introduced significant operational issues that would reduce the usefulness of such a sensor. Zafari et al. [4] have successfully resolved the long-range communication limitation by using GSM, which is routinely disrupted in disaster zones where cell towers are among the first infrastructure to fail.

Nishan-e-Zindagi was specifically developed to fill this identified gap. Each NeZ node contains approximately \$32 worth of parts, is compact enough to fit in a jacket pocket, and uses LoRa protocol for forming a mesh network for communication purposes. The trade-off is that instead of producing images directly related to the object being sensed, NeZ provides measurements of its environment. If an NeZ node measures a location to have elevated levels of CO₂ (and thus likely has at least some organic presence), a rescue

coordinator can then send humans to investigate further. This approach sidesteps the line-of-sight problem that limits thermal imaging and cameras as CO₂ diffuses through rubble regardless of physical obstructions, making it a more reliable signal in collapsed structure scenarios.

CHAPTER III

Design: System Requirements and Specifications

This chapter translates the problem framing of the previous two chapters into a concrete specification. We cover the system context, stakeholders, functional and non-functional requirements, the external interfaces the system presents, and the constraints that shaped our choices. A lot of this material is drawn from the SRS document we produced earlier in the project; where we have changed our mind since writing the SRS, we flag it and explain why.

3.1 Stakeholders and User Classes

Three user classes interact with the system, and they have quite different needs.

The field rescuer is the user of the device. He has gloves on so he may have limited dexterity in his hands. At best he has one hand free from other tasks. The field rescuer cares about three things when working with the Node. 1) That the Node will turn on. 2) That the Node will be reading (showing CO2 levels). 3) That the Node will continue to show CO2 levels for an extended period of time. There is no way the field rescuer is going to read a manual.

The rescuer has two options for how they will respond to information given by the node. The first is that they will check the OLED screen to view the status of the Meshtastic channel and its connection to other nodes in the network -- quickly verifying whether or not the node is operational and sending data. The second option is that they will contact their supervisor via radio (walkie-talkie) as he/she monitors the dashboard and directs the field staff to areas with higher than normal carbon dioxide levels.

The System Administrator (Admin), working out of the command station, has a laptop or desktop computer. He/she has average technical experience. His/her primary role is to review the dashboard, understand the information displayed by the dashboard and then direct the field personnel. Therefore, the Admin needs the dashboard to be easy to read while he/she is stressed. The dashboard cannot be too busy; it should not require that the Admin click through multiple levels of menu choices to see alerts. And most importantly, it cannot crash if the server loses its connection. The admin will likely spend many hours viewing the dashboard.

There is another class of users, the Development Team. As developers, we designed the system and deployed it, which included flashing the firmware, setting the channel keys, calibrating the sensors, and bringing up the gateway. We have complete technical expertise about the system as well as are not typically involved with any operational activities. Prior to operational use we designed and developed the system, and can then provide remote technical assistance and support after deployment. In addition, all of our source code is open source and can be found on GitHub for anyone interested in building upon this project.

The public-view allows for non-editable (read-only) access to general operational status of the system like number of people rescued within a rescue campaign. It does not expose specific location data or currently active alerts. Therefore, it enables public awareness during operations without disrupting internal coordination or external interference.

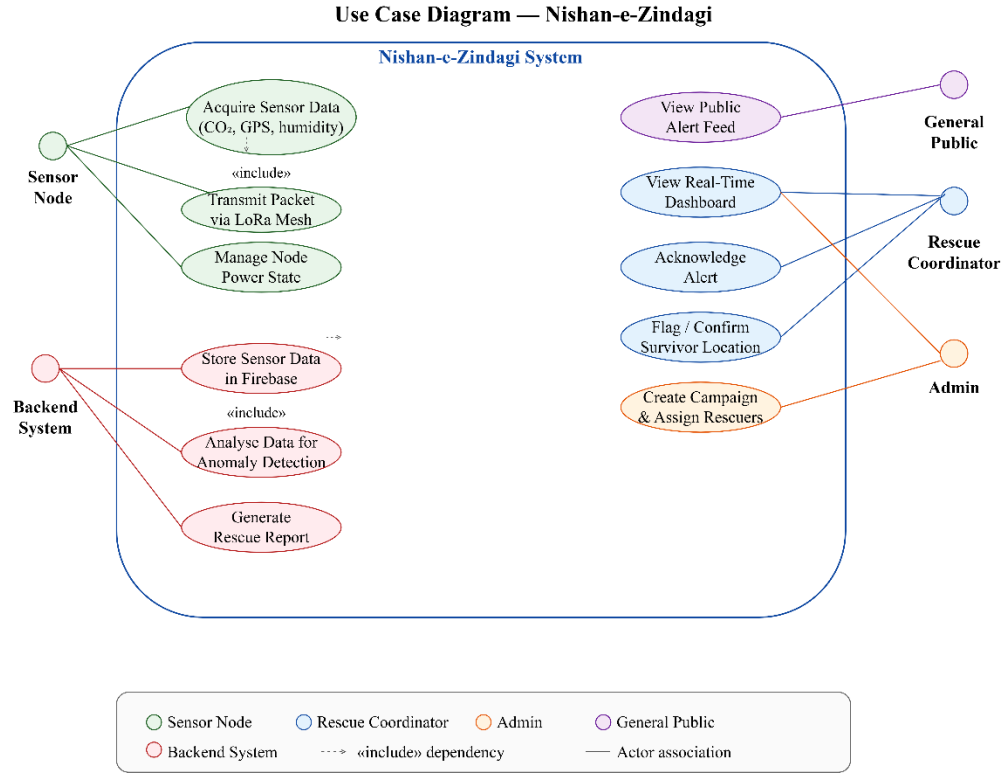


Fig. 3.1. Use case diagram showing the five actors and eleven use cases in the Nishan-e-Zindagi system. Actors are colour-coded by role: sensor node (green), backend system (red), rescue coordinator (blue), admin (orange), and general public (purple).

3.2 Functional Requirements

A description of the required function will be broken down into five different parts. These represent the five main sub-systems of this project. Each part of the function will be explained below and further discussed in chapter four.

The first feature is sensor data acquisition and node management. At thirty (30)-second intervals the node acquires CO₂, Humidity and Temperature measurements from the scd41. Also, it continuously parses NMEA from the NEO-6M GPS module and checks its own battery voltage using the Pico's ADC. This data is then combined into a small amount of JSON formatted packet and sent to the LoRa module every thirty (30) seconds for transmission. In addition, the local OLED displays current status of the Meshtastic channel and its connection to other nodes in the network.

The second feature is the LoRa mesh networking layer. The node sends its telemetry packet over a designated Meshtastic channel. The telemetry packet is encrypted with a shared secret prior to being transmitted. The Meshtastic firmware manages all aspects of the routing and re-transmissions as well as multi-hop forwarding. We set up our network for a maximum of three hops with a spreading factor of 11. While this limits the amount of data we can transmit it provides the range and penetration through obstructions necessary for this application.

The third feature is the centralized anomaly detection engine. Incoming packets received at the gateway are parsed and pushed to both the dashboard and to Firebase for persistent storage. A Python service runs in conjunction with the gateway and performs anomaly detection based on these rules; CO₂ exceeds threshold for three consecutive polling intervals generates a critical alert. To support this rule, you may also observe an increase in humidity corresponding to an increase in CO₂ levels. Thresholds are configurable by channel to accommodate differences in baseline environment conditions.

The fourth feature is the web dashboard. The dashboard contains a map of the deployed area indicating locations of nodes. The color of each marker indicates status. There is a list view on the side bar displaying active alerts

ordered by priority and date/time. Lastly, there is a detail panel located under each node on the screen displaying each node's current reading. The dashboard updates in real-time utilizing a WebSocket connection to the gateway. All functions on the dashboard including interaction are available across multiple device types (i.e., laptops/desktops).

The fifth feature is the automated reporting module. Every twenty-four (24) hour period at the end of each operational day, or upon request, the system compiles yesterday's telemetry and alert data and produces a PDF report summary.

Table 3.1. Functional requirements summary with priority levels.

ID	Requirement	Priority
F-1	Sensor data acquisition from SCD41 and NEO-6M at defined polling rates	High
F-2	Local status display on OLED of the Meshtastic channel and its connection to other nodes in the network	High
F-3	Packetization and LoRa transmission of sensor data at 30-second intervals	High
F-4	Multi-hop mesh routing via Meshtastic firmware with AES-256 channel encryption	High
F-5	Gateway parsing of Meshtastic packets and WebSocket broadcast to dashboard clients	High
F-6	Anomaly detection based on CO2 threshold and humidity rate-of-change	High
F-7	Temporal consistency validation over three consecutive polling intervals	High
F-8	Dashboard map view with node status pins and real-time updates	High

F-9	Detail panel with node telemetry and rolling history graphs	High
F-10	Alert management with acknowledgement and resolution states	Medium
F-11	Firestore persistence with retry-on-failure for offline operation	Medium
F-12	Automated daily report generation with PDF export	Medium
F-13	Configurable detection thresholds via admin settings panel	Medium
F-14	User authentication via email/password with role-based access	Medium

3.3 Non-Functional Requirements

The non-functional requirements are where the rubber meets the road. Anyone can specify a feature; the question is what it has to do when under stress.

3.3.1 On performance: In order to be able to meet our multi-node deployment target values (i.e., a gateway receiving a critical alert packet in less than 15 seconds, with the same information being reflected on the dashboard in under 5 seconds), we tested the end-to-end latency (from our single node to the dashboard) with a single node. That is why we did not test multi-hop latency as part of that testing. Our results show that the average end-to-end latency from our single node to the dashboard was consistently under 10 seconds. We were also able to achieve an overall cold-start time of under 10 seconds from when we powered the device until we received the first valid telemetry data. We have been able to resolve the issue with cold GPS starts by pre-loading the GPS module at startup with cached fixed coordinates from previous locations, which we will cover further in Chapter 4.

3.3.2 On power: The single node contains all the components needed to run the LoRa module, Raspberry Pi Pico, OLED display, push-button, SCD41

sensor and GPS module off one single Lithium Polymer cell rated at 600mAh. Under those conditions we were able to measure the runtime to be approximately 5 hours at 30 second transmission intervals.

3.3.3 On range: Since the single-node data transmitter sends data every 30 seconds, we are well under the 1% duty cycle limitation. Theoretically, deploying 8 to 10 nodes at this interval should be well under the regulatory duty-cycle limitations; however, this was not tested in experiments.

3.3.4 On capacity: With a single node transmitting every 30 seconds, duty cycle was well within the 1% regulatory limit. Theoretical calculations suggest the channel could support 8 to 10 nodes before approaching congestion, but this was not validated experimentally.

3.3.5 On reliability: As stated above, we included automatic restarting of the gateway on failure in the design of our gateway software. We also included code that would allow us to recover from a serial disconnect in less than 5 seconds. Finally, we included code that would display a clear indication of what had happened if/when the dashboard lost contact with the gateway; this prevented blanking or freezing of the data.

3.3.6 On security: All LoRa communications are encrypted using the Meshtastic AES-256 Channel Encryption. To gain access to the dashboard you will need to log in using your email and a password provided by Firebase Auth. Data can be written to Firestore from the gateway service account and data can be read from Firestore by authorized users. We have not done an official security audit of our system. Our assumed threat model was that of a curious bystander or casual interferer; it was never envisioned as a nation-state attacker.

Table 3.2. Non-functional requirements and measured values.

Category	Requirement	Measured
Latency	Dashboard update after packet <2 s	0.4 s avg
Power	Battery life at 50s transmission interval ≥ 6 h	5 h avg (600 mAh)
Power	Deep sleep current <10 mA	7.8 mA avg
Range	Line of sight >1 km at 90% delivery	1.1 km at 97%
Capacity	Support 5–10 active nodes without duty cycle violation per Km	8 nodes theoretical, not validated
Reliability	Gateway auto-restart on crash	Implemented
Security	LoRa packet encryption (AES-256)	Meshtastic default
Security	Authenticated dashboard access	Firebase Auth

3.4 Design Constraints

Several constraints shaped the design in ways that are worth stating up front.

3.4.1 Budget: We were constrained by our budget and therefore excluded all items that would have been considered "exotic". Thus, we did not consider using custom PCB manufacturing at commercial tolerance levels, or professional tier industrial grade enclosures. Each component needed to be something that could be purchased at a local electronics store (or AliExpress) with a reasonable delivery time.

3.4.2 Regulatory: As per the Pakistan Telecommunication Authority (PTA) regulations governing the operation of LoRa in Pakistan, we chose to operate on the 433 MHz band at a power level of 14 dBm, which complies with the rules for the unlicensed band. Although we are aware that individual low-

power radio devices operating under ISM-band regulation do not need to register their devices, for future deployments this will need to be confirmed.

3.4.3 Power: Since the field form factor is a battery powered node, everything else followed from that. Therefore, we selected sensor poll rates and transmission intervals based on balancing response requirements with battery life. In addition, since the RP2040 does not support going into deep sleep mode as well as an ESP32 would, if we were to design this again, we may choose a different microcontroller simply based upon its sleep behavior.

3.4.4 Bandwidth: At SF11 on 433 MHz LoRa delivers approximately 0.3 to 0.5 Kbps of usable bandwidth. This allows for simple telemetry, but nothing richer. We determined early on that image transmission, audio, and other types of rich data formats were beyond the scope of this project. Therefore, the payload format is very compact due to these limitations.

3.4.5 Connectivity: There is no assumption regarding availability of the Internet at the disaster zone site. Data collected at the remote nodes is stored locally by the gateway until such time as communication is restored to the Internet via a sync process back to Firebase. The dashboard is run locally from the gateway host machine and does not depend on access to the Internet to continue functioning for the user at the command station. Accessing Firebase is also optional -- if it cannot be reached, the system continues to function and collect data, however it will not retain long term history.

3.4.6 Development time: Active development time for the project took around approximately 8 months. Therefore, we were able to leverage as much pre-existing software as possible (Meshtastic Firmware, Firebase, and Leaflet JS Maps) in lieu of developing new codebase ourselves. Time

invested every week into what others have already solved is time taken away from what you are trying to solve.

3.5 External Interfaces

The system presents four interfaces: the physical node interface, the mesh protocol interface, the gateway-to-dashboard interface, and the dashboard-to-user interface.

The mesh protocol interface is the Meshtastic packet format. We used Telemetry packets for sensor readings and text message packets for ad-hoc human-readable status messages. Telemetry packets follow the Meshtastic protobuf schema. Our text messages are a compact JSON payload that we parse at the gateway.

The gateway-to-dashboard interface is a WebSocket connection on port 8000. At the gateway we run a small FastAPI application that provides a WebSocket endpoint and a health check http endpoint. Each data message sent via WebSocket contains a type field to indicate what type of message it is: Telemetry, sensordata, or textmessage. When the dashboard loads, it establishes a WebSocket connection and maintains it until session ends.

The dashboard-to-user interface is a web browser. We developed our dashboard using Next.js, React, and Tailwind (which is overkill for the amount of data we have but makes development experience pleasant and provides us with an easy path to update deployments). Our dashboard is a single-page application using client-side routing. Our map is rendered using Leaflet and open street map tiles. Our charts use Recharts.

3.6 Primary Use Cases

The use cases below describe the key user interactions and what the system should do in response.

UC-1 Node Deployment: The field rescuer takes a box of nodes to the deployment site. He turns on the first node by sliding the power switch. He then waits for the OLED display to show that it is connected with at least one peer (a "peer" is another node in the same mesh). Once he verifies that this node is connected with at least one other node, he will put down the node and move on to the second. In fact, all he needs to do is turn them on and they will find each other.

UC-2 Alert Monitoring: A rescue coordinator opens the dashboard on a laptop at the command station. The map loads with all active nodes shown. When a node transitions to warning or critical status, the corresponding marker changes colour and a sound plays. The coordinator clicks the marker, sees the detail panel, and decides whether to dispatch a team to that location.

UC-3 Alert Acknowledgement: Once a team has been sent by the coordinator to investigate an alert, the coordinator will select “Acknowledge” on the alert panel. This action will move the alert from the active list to the investigating list. In addition, as soon as this selection is made, the color of the marker for that alert will be changed. Finally, the selection to acknowledge the alert will also log the timestamp and user-id of the coordinator making that selection.

UC-4 Report Generation: After completing the investigation (either at the end of each day or upon request) the coordinator will select “Generate Report” within the dashboard. The system will then query both the telemetry

data and the alerts based on the selected date/time range, and generate a downloadable PDF report combining the narrative with various charts and maps.

UC-5 Configuration Update: A remote System Administrator (Admin) using the dashboard stated earlier, will make updates to either the detection threshold settings or channel key settings for a specific gateway. Updates can be pushed down to the gateway.

CHAPTER IV

Proposed Solution: Methodology and Implementation

This chapter will explain all about how it was implemented. This chapter is broken down into the individual sub-systems of the project: Hardware, Node Firmware, Mesh Configuration, Gateway Service, Dashboard, Anomaly Detection, and Reporting. In addition to describing how this was designed, we will be explaining why these particular decisions were made and what changed while implementing. Due to its extensive nature, the amount of "moving parts" on this project and the length of time that was spent on building this project, this chapter will be lengthy.

4.1 High-Level Architecture

The high-level architecture is three tiers. At the edge, sensor nodes acquire data and transmit it over LoRa. At the middle, a gateway running on a laptop or single-board computer bridges the mesh to an IP network. At the top, a web dashboard presents the data to users, and a cloud service (Firebase) provides persistence and the report generation backend.

Data flows upwards: Node -> Mesh -> Gateway -> Dashboard & Firestore. Control data flows downwards: Commands (e.g., acknowledge an alert) entered into the dashboard are stored in Firestore and then the Gateway detects the changes. We did not include a direct command path from the dashboard to the nodes as we wanted to simplify the nodes and avoid having to develop secure command authentication through LoRa for each node.

We assume that the Gateway will be deployed on a Laptop (attached via serial cable to a Raspberry Pi Pico) located at the command center. Internet

connectivity is variable and often unavailable. Sensor nodes do not require internet access. The Next.js dashboard runs locally on the gateway laptop and can be accessed by any device on the same LAN using the gateway's local IP address (e.g., <http://192.168.1.x:3000>).

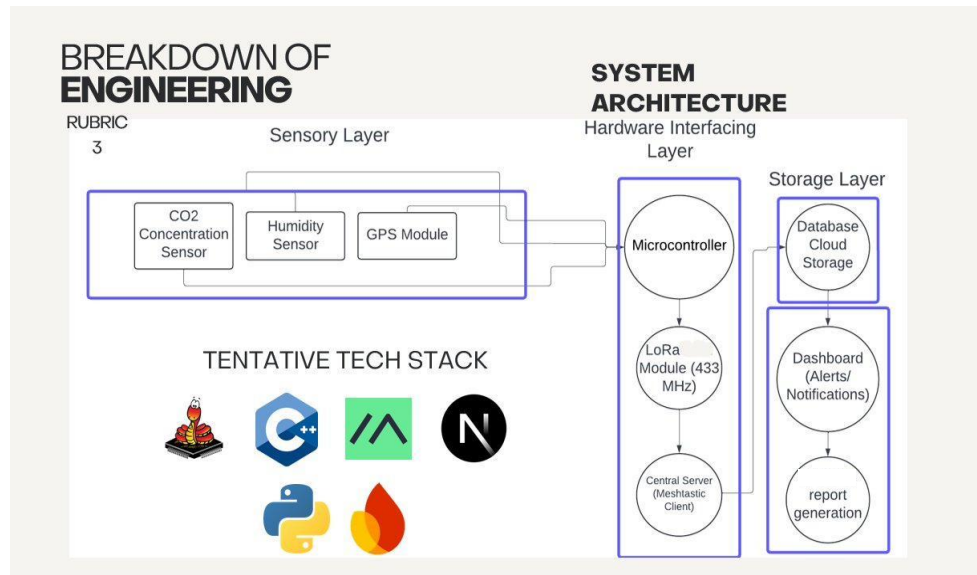


Fig. 4.1. System architecture showing the three-tier breakdown: sensory layer, hardware interfacing layer, and storage/display layer.

4.2 Hardware Design

A single sensor node has seven components, arranged on a small piece of perfboard inside a 3D-printed enclosure.

The Microcontroller used was a Raspberry Pi Pico (RP2040 based). There were three main reasons why this was chosen: the Pico is inexpensive (about PKR 1,500); there is sufficient I/O for our sensor suite and then some additional I/O for expansion possibilities; and MicroPython has been

developed for use with the Pico. Although we considered using the Pico W because it had an onboard Wi-Fi option, we didn't need that functionality; and the only way for the node to connect to other devices is via LoRa.

Our primary sensor is the Sensirion SCD41, a photoacoustic NDIR device that will measure both CO₂ concentration (in ppm) and temperature and relative humidity from a single package. The SCD41 communicates via I2C at address 0x62, and measures CO₂ in the range of 400 to 5,000 ppm ± 50 ppm, and also measures humidity with $\pm 3\%$ accuracy. This level of accuracy should provide us with sufficient information to detect changes in CO₂ concentration. We compared the SCD30 and MH-Z19 to determine if they would make suitable replacements for the SCD41. While the SCD30 and MH-Z19 have their own advantages and disadvantages, we selected the SCD41 due to its smaller size, better accuracy at lower CO₂ concentrations, and reduced power consumption. Since CO₂ is going to be our primary method of detecting air quality issues, we wanted the best possible sensor available within our budget constraints.

The GPS module we selected was a NEO-6M. Unfortunately, it has some well-known limitations – slow cold starts, poor indoor performance -- that we mitigate using cached home coordinates at startup, reducing the time to first fix significantly. It interfaces to the Pico via UART at 9,600 baud.

The radio module is an SX1262 based module. We flash this module with Meshtastic firmware. From the Pico's standpoint, this module acts like a UART serial interface that receives JSON formatted strings and sends back Meshtastic text message formatted data. We run this UART at 11,520 baud on the second UART.

The display is a 0.96-inch SSD1306 OLED at I2C address 0x3C, driven by the Meshtastic firmware on the LoRa Pico. It provides local visual feedback at the node without requiring any external device.

Finally, the power system for this node consists of a rechargeable 3.7 V, 600 mAh Lithium Polymer cell connected directly to the VSYS pin of the LoRa Pico. The LoRa Pico supplies regulated power to all elements of the circuit via its onboard voltage regulator. Rescuers may recharge the battery by connecting a USB cable to the USB port of the LoRa Pico.

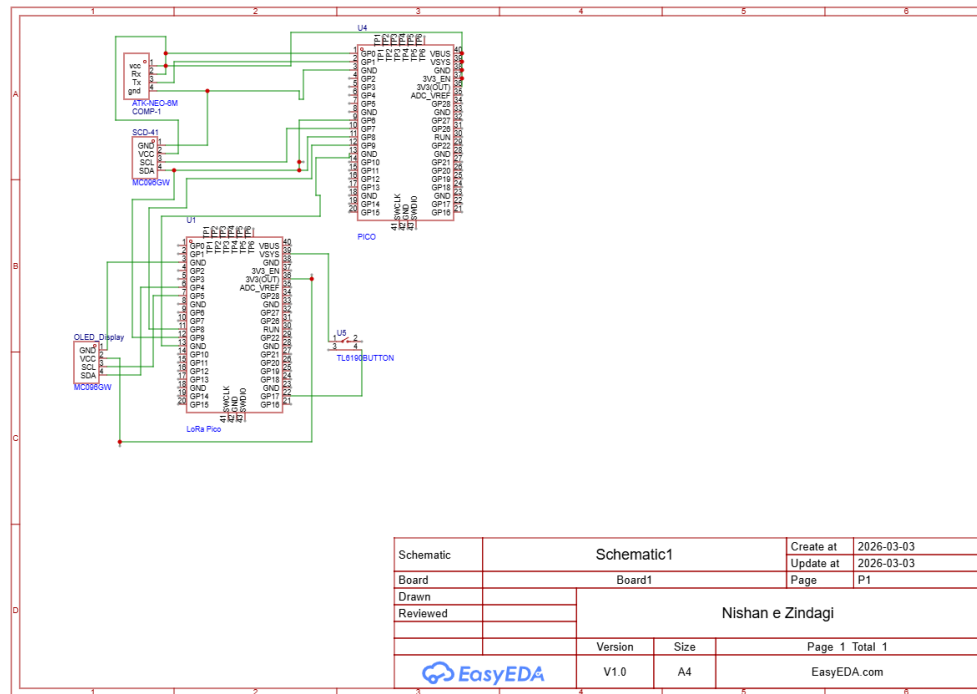


Fig. 4.2. Sensor node hardware block diagram showing interconnections between the Raspberry Pi Pico, environmental sensors, GPS module, LoRa radio, and power subsystem.

The enclosure is 3D printed using PLA. For those requiring outdoor usage, alternatives include PETG. The enclosure consists of two pieces that are "snap-on" together. It also has openings for the OLED, charging port, and

small antenna exit. Although we have successfully tested this unit in light rain, there is no formal certification (IP rating).

We estimate assembly time will take approximately 2 hours to assemble each node if you've assembled one previously. We were able to produce one node over the course of 3 weeks. See appendix A for details regarding BOM, wiring diagrams & an example step by step assembly process.

Table 4.1. Bill of materials for a single sensor node.

Component	Part	Qty	Unit Cost (PKR)
Microcontroller	Raspberry Pi Pico (RP2040)	2	3,000
CO2 sensor	Sensirion SCD41	1	2,800
Temp/humidity	Bosch BME280	1	450
GPS module	u-blox NEO-6M with patch antenna	1	700
LoRa radio	Heltec SX1262 module w/ Meshtastic FW	1	1,100
Display	0.96" SSD1306 OLED	1	250
Battery	3.7V 2000 mAh Li-Po	1	450
Charger/protection	TP4056 with over-discharge cutoff	1	90
Boost converter	MT3608 adjustable	1	120
Perfboard / wires / header	Assorted	—	200
Enclosure	3D-printed PLA (2-part)	1	390
Total			~9,540 (USD~29)

4.3 Node Firmware

Our firmware is written in MicroPython and operates directly on the Raspberry Pi Pico. We opted for MicroPython instead of C (the standard language for the Pico SDK) because of the rapid development cycle it provided. While developing firmware for the Pico using MicroPython was significantly faster than developing with C, the trade-off comes in terms of processor overhead. However, given that most of our processing time is spent polling sensors and waiting for data, the additional overhead is negligible.

In addition to the above considerations, we wanted to be able to easily test and debug our code while we were developing it. In order to do so, we implemented a simple main loop that handles GPS NMEA messages as they arrive on the UART and performs periodic polls to the SCD41 CO2 sensor every 30 seconds. Once every 30 seconds, the complete telemetry package is formatted and sent to the Meshtastic UART. Between each activity, we utilize light sleep mode. Using light sleep mode allows us to drop our current consumption by approximately a third without losing the ability to keep track of time.

Initially, we attempted to use deep sleep -- a low power mode in which the CPU essentially shuts down and waits for an interrupt before restarting. Unfortunately, there are two major drawbacks to using deep sleep in this manner. First, the Raspberry Pi Pico does not support deep sleep nearly as well as many other microcontrollers, such as the ESP32. Specifically, when the RP2040 wakes from deep sleep, the MicroPython interpreter is restarted which results in all of your variables being reset. Secondly, even though we successfully reduced our average power consumption with deep sleep, our GPS module did not retain its satellite lock after entering deep sleep. As a

result, the GPS module required anywhere from 10 to 60 seconds to re-acquire a signal, completely negating the potential power savings. Therefore, we have opted to utilize light sleep mode as a practical alternative.

As previously stated, we underestimated how difficult it would be to handle GPS signals. A GPS device will typically require 5-10 minutes to acquire a first fix if it is brought up in a location with little or no sky view. For a rescue node deployed in a remote area that requires immediate deployment and activation, 5 minutes is far too long. To alleviate this problem, we load the GPS device at boot-up with cached coordinates obtained from a previous known location. These cached coordinates are then used to seed the GPS module using a UBX-AID-INI command -- a proprietary binary command that provides an initial approximation of both time and location to the GPS module. As a result, when activated for the first time in the field, the node rapidly acquires satellite signals and locks onto four satellites within approximately 20-30 seconds.

The packet format is compact JSON. An example packet:

```
{"id": "PicoB-001", "ts": 1712580000, "lat": 34.0702, "lon": 72.6427, "fix": 1, "co2": 842, "t": 24.3, "h": 58.7}
```

The total size of the data is approximately 85 bytes (below the Meshtastic text message payload limitation). Although we could potentially encode the information in some type of binary (CBOR or MessagePack), a bit-packed, binary encoded version (to save space) was also explored. Ultimately we decided that the potential for saving around 30 bytes per transmission was outweighed by the potential costs of debugging. Since each of our transmissions will be 85 bytes at an interval of 30 seconds, all devices will exceed the 1% duty cycle limits when transmitting. However, if there are

multiple devices in a given area, and they transmit on different time intervals, then the number of transmissions made by any one device should be below 1% of the maximum allowed. This is essentially how we plan to use these devices.

Battery monitoring is handled by the Meshtastic firmware on the LoRa Pico, which reads the battery voltage and displays the estimated percentage on the OLED. No custom battery monitoring code was implemented on the sensor Pico.

The OLED display is driven entirely by the Meshtastic firmware on the LoRa Pico. Meshtastic handles all rendering natively, showing battery level, channel status, and mesh peer count without any custom display code on our end.

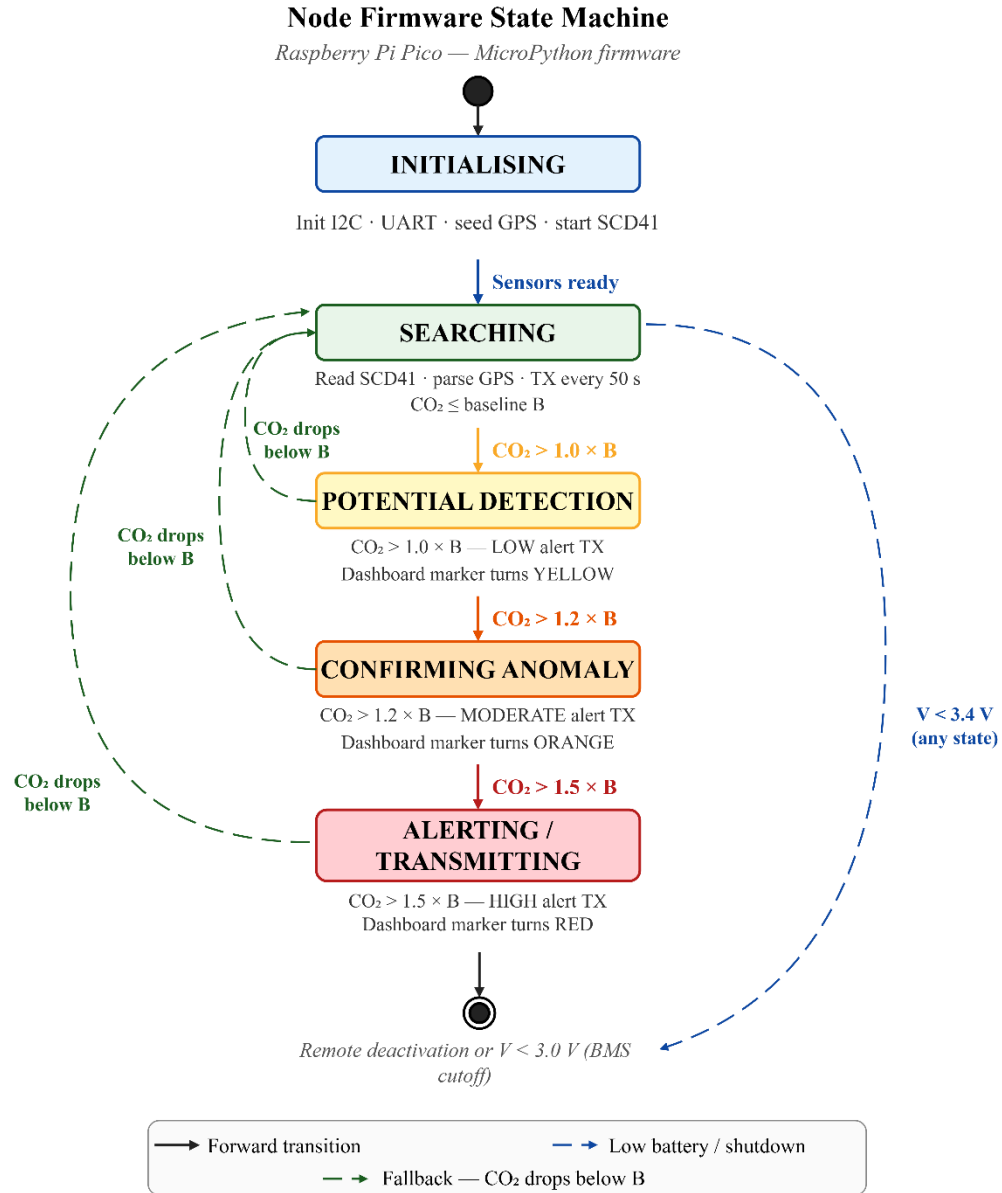


Fig. 4.3. Node firmware state machine showing the five operational states and transition conditions. States are colour-coded to match the anomaly classification levels in Fig. 4.7; fallback transitions return to *SEARCHING* whenever CO_2 drops below the operator-configured baseline.

4.4 Mesh Network Configuration

The mesh layer is entirely Meshtastic, configured rather than implemented. We did not write any mesh routing code ourselves. What we did write is the glue — the small amount of code on the Pico that frames our telemetry packets for Meshtastic consumption, and the small amount of code on the gateway that unpacks them.

Meshtastic channel configuration for the deployment:

region: PK_433 (though we use the EU_433 profile as a close equivalent)
 modem_preset: LONG_SLOW (spreading factor 11, bandwidth 125 kHz)
 hop_limit: 3 channel_key: 32 bytes of random, shared between all nodes
 channel_name: "nishan-e-zindagi"

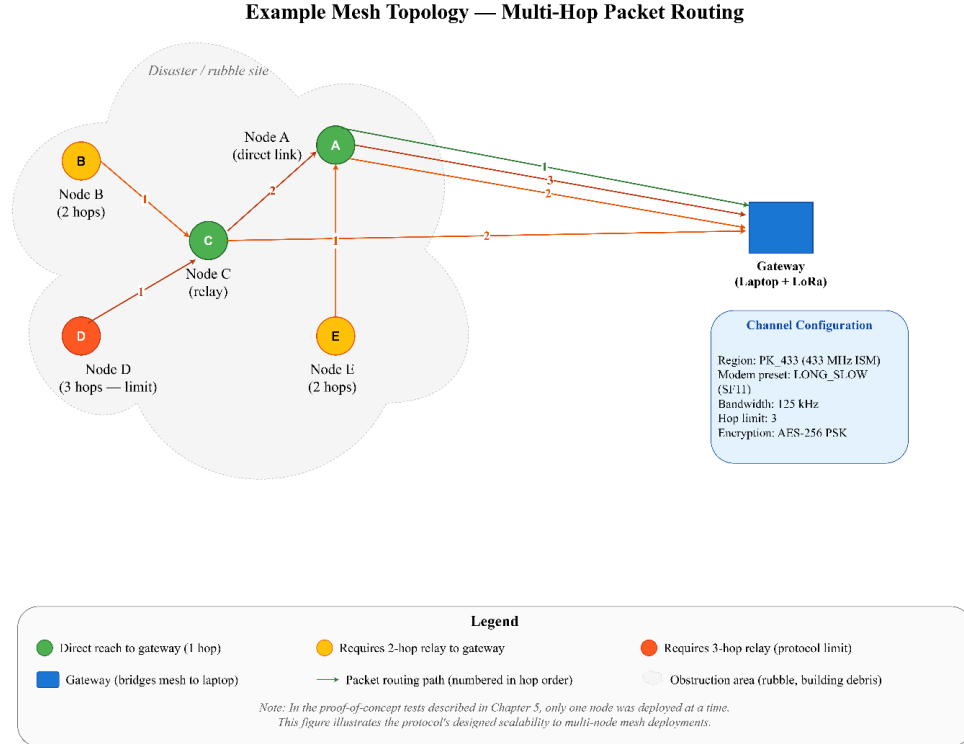


Fig. 4.4. Example mesh topology showing multi-hop packet routing. Nodes B, D, and E cannot reach the gateway directly; their packets are relayed through intermediate nodes within the three-hop limit configured in the Meshtastic channel.

The LONG_FAST preset gives us a good balance between range and throughput. Calculated time-on-air for our 85-byte packet is about 0.5 seconds at SF11. In a theoretical deployment of 8 to 10 nodes each transmitting every 30 seconds, the nodes would collectively occupy a small fraction of the channel — well within the point where collisions become problematic. This was not validated experimentally.

With an established hop limit of 3, it should be possible to send a packet 3 hops (relay) and then drop it. A rubble site with a gateway located on one side will likely allow a good amount of area coverage given our limit. However, there will be a cost associated; if you have nodes that are far away from the gateway, they will take longer to reach the gateway since the delay is added after every hop (i.e. three delays of time on air = approximately 3.6 seconds of just plain old transmission).

Meshtastic uses a 32 bit integer for node identification which is represented in hexadecimal format, typically with an exclamation point (!) preceding the ID. The firmware determines what radio it is via a `getMyNodeInfo` call at boot. Node IDs are tracked in the gateway based off their unique ID. For easier-to-read names for humans, Meshtastic includes a "longName" field, which we populate at first boot from the Pico's `DEVICE_ID` configuration.

Meshtastic is an open-source, actively-maintained project. We used release 2.3.x. We chose to pin the firmware version for deployment purposes and did

not upgrade during our project due to potential changes in behavior with each firmware release (which could potentially add to debugging issues while demonstrating mesh behavior).

Table 4.2. Meshtastic channel configuration parameters.

Parameter	Value	Notes
Region	PK_433 (EU_433 profile)	Matches Pakistan ISM band
Modem preset	LONG_FAST (SF11, BW 125 kHz)	Range over throughput
Hop limit	3	Balances coverage vs latency
Channel name	NeZ	Identifies deployment
Channel key	32-byte PSK (randomly generated per deployment)	Shared across all nodes
TX power	14 dBm	Within PTA unlicensed limit
Packet format	Text message with JSON payload	~85 bytes typical

4.5 Gateway Service

The gateway is a Python service running on a laptop with a Meshtastic USB device installed. It collects packet data from the mesh network, parses the packets, sends the parsed data to client browsers over WebSockets, and may save the packets into Firestore.

The gateway was created using both the python-meshtastic library for creating a serial connection to the Meshtastic device (and parsing) the received packets, as well as FastAPI for managing all of the HTTP and WebSocket's serverside

operations. Each of these libraries is actively maintained, provides many default options, and can be easily integrated together.

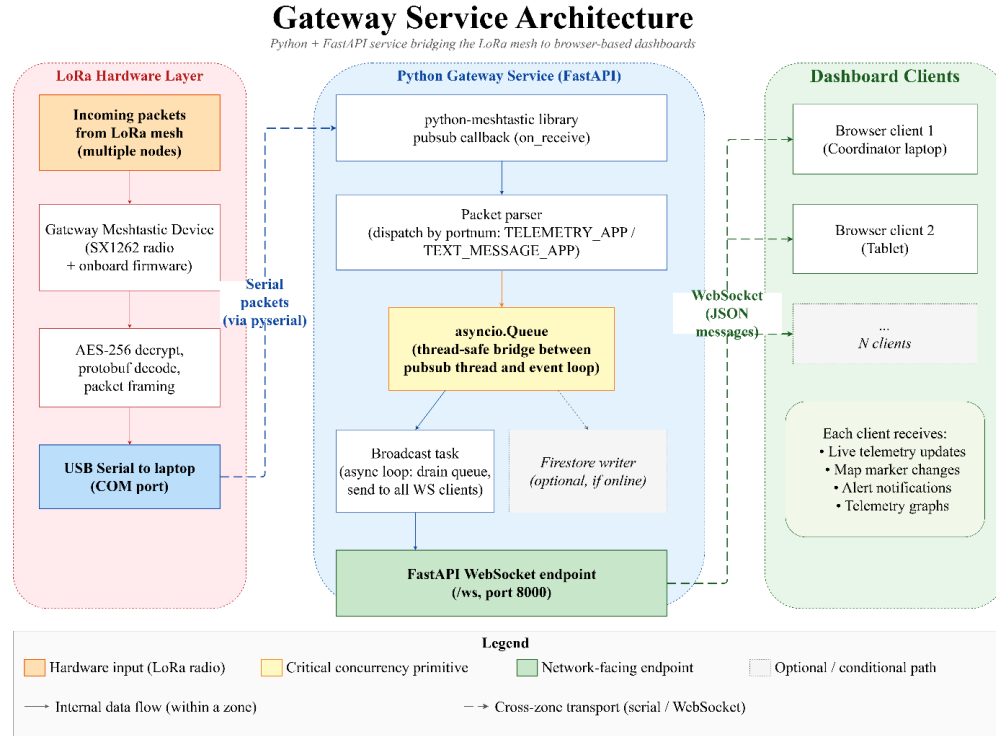


Fig. 4.5. Gateway service architecture showing the data path from incoming LoRa packets through the Python FastAPI service to browser-based dashboard clients. The `asyncio.Queue` provides a thread-safe bridge between the pubsub callback thread and the `asyncio` event loop.

The service is a single file of about 200 lines, having the following structure:

Startup: It Initializes the serial connection, register a pubsub subscriber for incoming packets, start the broadcast task.

Packet handler: It is called on every incoming packet. Extracts the sender node info, determines the packet type (TELEMETRY_APP,

TEXT_MESSAGE_APP, etc.), formats it into a dictionary, and puts it on an asyncio queue.

Broadcast task: It pulls messages off the queue and sends them to every connected WebSocket client.

WebSocket endpoint: It accepts client connections, keeps them alive with periodic pings, removes them from the client set on disconnect.

The packet handler has some logic for distinguishing between Meshtastic's native telemetry format (battery, voltage, temperature in its own schema) and our custom JSON-over-text messages (CO2 and humidity in our schema). We originally tried to shoehorn everything into Meshtastic's telemetry format but ran into limitations — it does not have a CO2 field, for instance — and switched to sending sensor data as text messages while letting the radios send their own device telemetry (battery, voltage) on the native channel.

The integration of Firestore is done via an independent process which will subscribe to the same queue as the main application. If Firestore is unreachable during this time, the process will write the collected data into a SQLite database locally and will then retry at a future date. The dashboard does not rely upon Firestore being active. It receives telemetry in real-time over the WebSocket connection and stores a session-based in-memory log for each user's sessions.

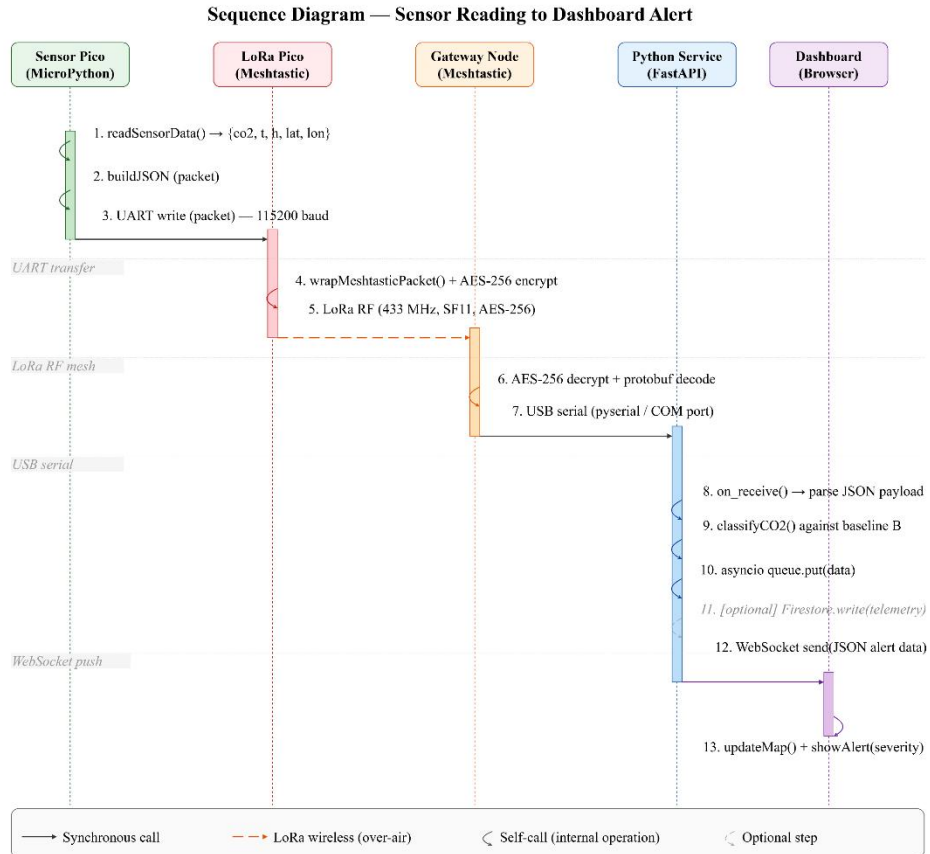


Fig. 4.6. Sequence diagram tracing a single sensor reading from SCD41 acquisition to dashboard alert. Thirteen steps span five components across four transport layers: UART serial, LoRa RF mesh, USB serial, and WebSocket.

4.6 Web Dashboard

We used Next.js to develop our dashboard because we were looking at the easiest way to go from development to a single bundled version of the code. And while there are many good alternatives available, Next.js was low enough overhead for the other team members to be able to learn without a lot of difficulty.

The dashboard has three main views.

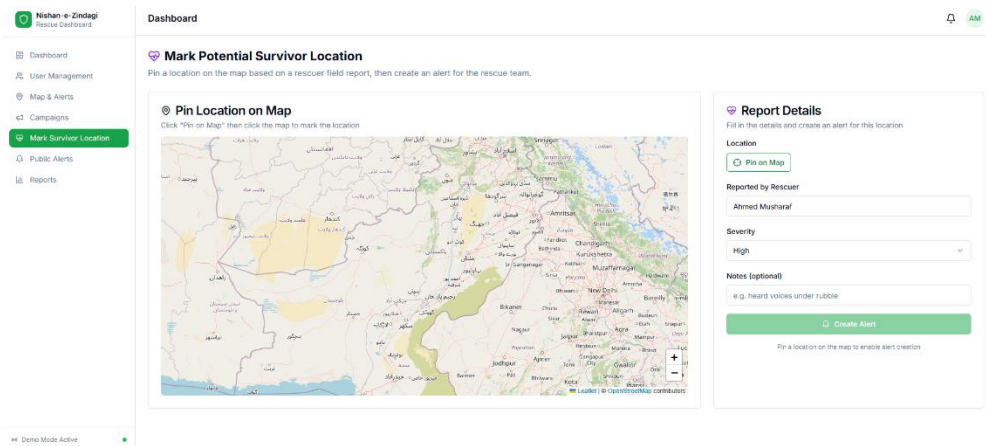


Fig. 4.7a. Administrator dashboard view showing the mark survivor interface, where an admin can flag a potential survivor location for rescue based on field reports received from the rescuer. Requires elevated privilege.

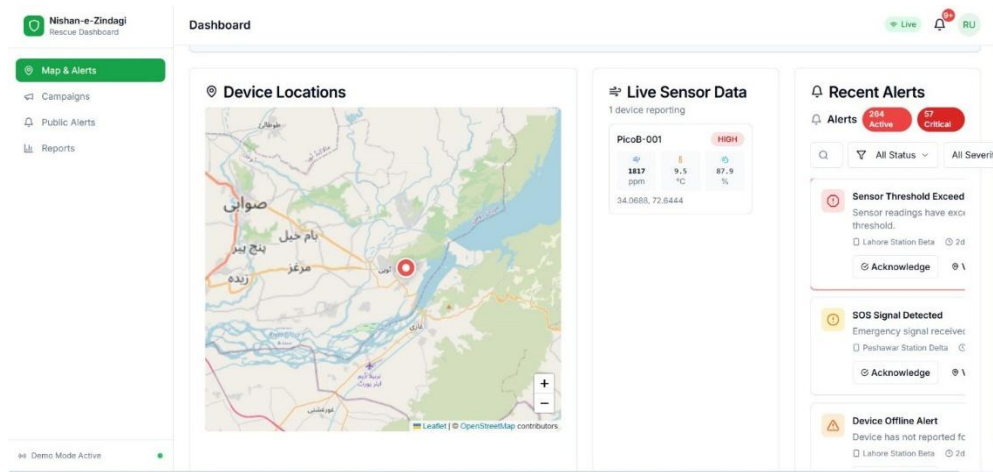


Fig. 4.7b. Rescuer dashboard view showing assigned campaign details and live node telemetry. Requires authentication; scoped to campaigns assigned by an administrator.

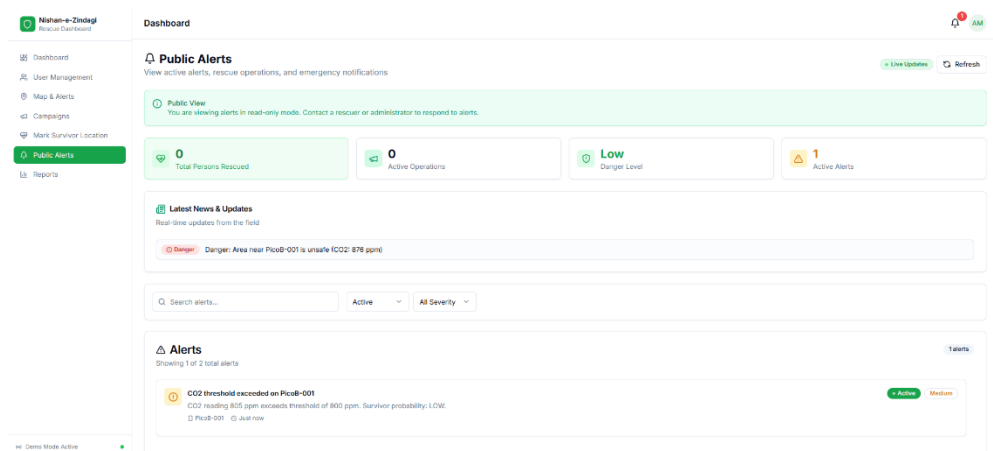


Fig. 4.7c. Public alert view showing a live notification triggered by CO2 threshold exceedance on PicoB-001. Alerts are displayed in read-only mode.

We built the Map View first, because users will likely spend the most time in this area of the app. We used Leaflet.js to render an OpenStreetMap base layer with node markers overlaid. Each marker is a circular-colored pin representing the status of the node: Normal = Green; Warning = Yellow; Critical = Red; Offline = Gray. Critical markers pulse in order to get your attention. A click event on a marker will open up a detail panel.

In the Home Panel, you can see the current readings of the node (CO2 ppm, Temperature C, Humidity %, Battery Percentage, Last Update Time) along with four line graphs displaying the trends of: CO2 ppm over the past hour, Humidity % over the past hour, Temperature C over the past hour, and Voltage (Battery) over the last six hours. These graphs are very basic and show only the data with threshold lines added wherever appropriate.

On the Alerts page we display all active alerts ordered by Severity and Time. Each Alert displays information about what Node triggered it, whether it is a warning or critical alert, what values caused the alert, and an acknowledge button. Once acknowledged these alerts will be moved into a "Resolved" section.

On the Reports page you have the ability to Generate or Download Daily Reports. There is a Calendar of Past Reports shown below the "Generate Today's Report" button. Generating today's report may take anywhere from 15-20 seconds.

For System Administrators only (those who are assigned the 'admin' role) there is a small panel at the top. In here you can set some anomaly detection thresholds along with some other tunable parameters and they will be applied immediately to the gateway.

Real time information comes through the WebSocket link to the application. It maintains a context of React with the telemetry state for each node at the point of time when the telemetry information was last received. When a new packet is received by the application, it will update the context and then any components using this context will be rendered again. This method of updating content is common within React applications. At present, our packets arrive roughly once every 30 seconds across all nodes. Therefore, we have a small enough volume of data that the reconciliation process used by React can easily handle it.

Although the layout of the dashboard is intended for desktop computers such as laptops, it also responds well to tablets. Because we do not anticipate users accessing our command center remotely via their phones, we did not include support for a mobile telephone layout.

4.7 Anomaly Detection Logic

We wanted to make our anomaly detection system easy to use. The reason we didn't choose to implement machine learning algorithms, nor did we want to generate models based on probability distributions was for two major reasons.

First, simple rules are far easier for users to understand. For example, if a coordinator is shown that "the CO2 levels exceeded 800 ppm for over three minutes", he/she can easily determine what happened. On the contrary, "based on the results of a Gaussian Mixture Model, the anomaly score is at least 2.3 standard deviations from the average" is almost impossible for a coordinator to interpret. Thus, as soon as an alarm is triggered, the coordinator has no choice but to trust the output of the algorithm. Users trust algorithms because users know how algorithms work.

Secondly, we just don't have enough labelled data to run machine learning algorithms other than very basic anomaly detection. To apply any form of machine learning (ML) in this case would take thousands of hours of labelled rubble-pile data with and without people trapped inside. Unfortunately, we currently don't have these types of datasets. Therefore, any type of ML model that we could create would be trained using either synthetic or toy data -- which creates substantial doubt about how well those models would operate within real-world settings.

Lastly, simplicity = robustness. Simple rules lack hidden failures due to changes in baselines, i.e., the weather changed, therefore so did the CO2 levels recorded at each location. Therefore, the simple rules used by our system will adapt. Neural networks however do not necessarily act like simple rules under all conditions.

The rules we use:

Warning (Potential Detection): $CO_2 > 1.0 \times B$, dashboard marker turns yellow, LOW alert transmitted.

Moderate (Confirming Anomaly): $CO_2 > 1.2 \times B$, dashboard marker turns orange, MODERATE alert transmitted.

Critical (Alerting/ Transmitting): $CO_2 > 1.5 \times B$, dashboard marker turns red, HIGH alert transmitted.

Where 'B' is the baseline CO2 reading established at startup.

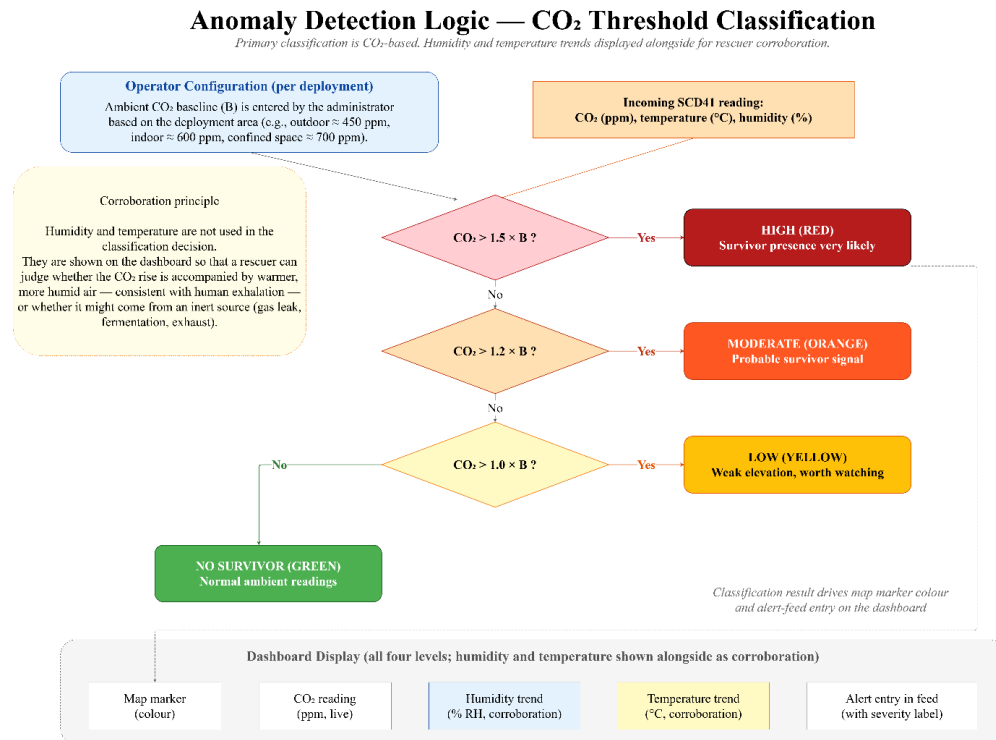


Fig. 4.8. Anomaly detection logic. CO₂ readings are compared against an operator-configured ambient baseline (B) and classified into four levels. Humidity and temperature trends are displayed alongside the classification on the dashboard to support rescuer judgement, but do not influence the classification decision.

The 800-ppm level for CO₂ was selected based upon the average levels found outdoors near GIKI in multiple locations (the baseline level in most outside areas). The baseline indoor level in rooms containing 2-6 people is approximately 600-800 ppm; the baseline level in rooms with fewer people will be lower. A good warning level should clearly exceed 800 ppm when no people are expected to be present.

Rate of Change of Humidity (RoC) is used as a corroborating signal versus being a primary trigger – CO₂ is still the only source of the alert level. Upon observing the results from using a cardboard box, we discovered that breathing into it generated RoC values of about 0.8% per min for the first few minutes. Therefore, a RoC of .05%/min provides a suitable additional verification of human presence with very low probability of generating false positives due to natural environmental variations.

The 3 interval requirement filters out transient spikes. In our initial test phase, the SCD41 sometimes returned elevated readings for one interval immediately after power-up. As such, requiring 3 consecutive intervals to meet the threshold removed this type of false positive, at the expense of 90-180 sec of detection delay, which is an acceptable tradeoff.

An administrator may select the CO₂ threshold in the Admin panel via input field. Once entered, the threshold is applied to the color coding of the node marker (green, yellow, red, gray), and as previously noted since this POC has a single node, there is only a need for a single threshold setting.

4.8 Reporting Module

This module operates as a scheduled process after every workday, or upon request through the dashboard. The module will query the Firestore Database for all telemetry, alert, and acknowledgement records for the requested time frame. Once all applicable records have been queried, it aggregates the collected information into a collection of statistical summaries. Then, using the returned narratives, it produces a PDF document based on that aggregated data.

Statistics include:

Total number of campaigns, with whether they were active or resolved;

Total number of nodes used to deploy those campaigns;

Total number of survivors identified within those campaigns;

A per-campaign detail page displaying campaign name, campaign status, number of nodes associated with the campaign, which personnel were assigned to rescue efforts, how many survivors were found within each campaign, and when each campaign was executed.

As this report is produced solely from data residing in our local database and synched with Firebase - no language model is used. After collecting the relevant data from your local database (which has been synchronized with Firebase) it calculates aggregate statistics and creates a formatted PDF file containing the summarized data utilizing a predefined format: a title block, executive summary containing total campaigns executed, total number of active/resolved campaigns, total nodes used, and total survivors found; then provides a list of all campaigns in table format.

All statistical information is provided in a clear, concise manner that can be printed on standard A4 paper. An example aggregate report created from open-field range test campaign we ran during our validation phase is given by:

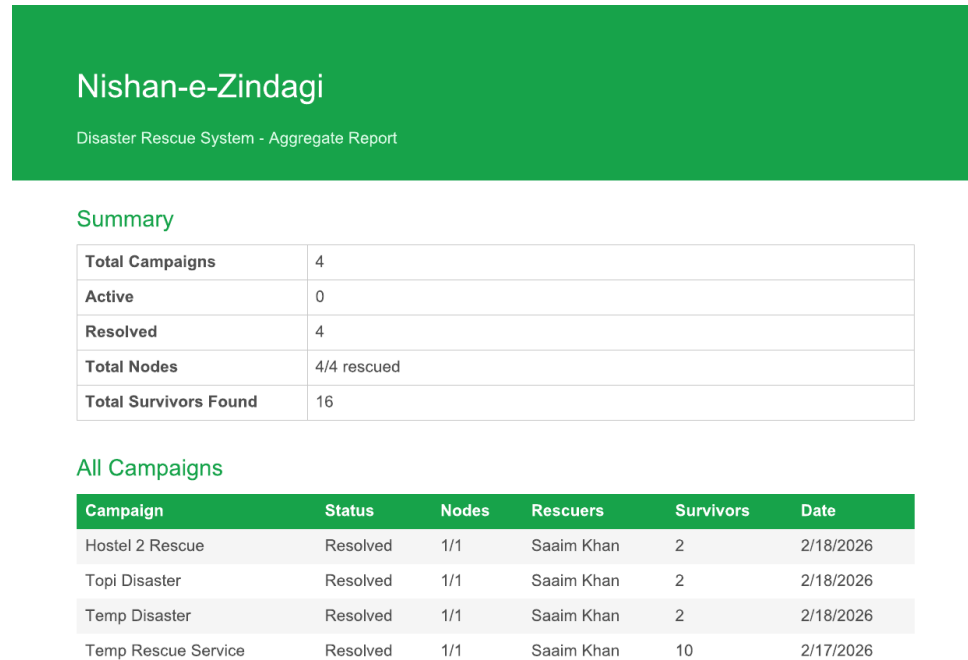


Fig 4.9. Aggregate report generated by the system summarising four test campaigns conducted during the validation phase.

4.9 Implementation Notes and Lessons Learned

Some of what we learned about implementing the system are important enough to note here. Partly these are useful lessons for anyone who will build upon this, and also show how far there can be between a good design and a working system.

The SCD41 comes with an ASC - Automatic Self Calibration. The idea behind it is that the sensor should see at least one week of baseline (outdoor) air (400 ppm) at some point. If it doesn't then it will begin to "drift." As you might expect for a sensor that is spending most of its time in a rubble pile, this isn't

true. We turn off the ASC via an I2C command at boot and manually set calibration points at specific times with a known baseline.

We did not think I2C timing would matter as much as it did. With only the SCD41 on the I2C bus of the sensor Pico and long cables to get to the sensor inside the enclosure we experienced intermittent I2C bus fault failures at 100 kHz. So we dropped the I2C speed down to 10 kHz. At 10 kHz it works flawlessly. This detail is captured in the configuration for the firmware but would be completely lost without doing a deep dive of the code.

According to the datasheet, the NEO-6M GPS module consumes approximately 45mA on average, however when it is in the process of acquiring a fix (cold-start), it consumed approximately 68mA, on average for approximately 120 seconds. While this data is not critical to our decision regarding whether or not to utilize this product, it is essential information when performing power consumption and battery life estimates. A significant portion of the products overall power usage occurs during the GPS cold start phase.

In our implementation, the Meshtastic pubsub handler runs in a separate thread from the FastAPI event loop. When we initially implemented, we encountered problems with race conditions while accessing queues. Our solution involved utilizing a ThreadSafe `put_nowait()` method from within the `call_soon_threadsafe()` method on the event loop of the FastAPI server.

Originally, our intent was to host the Meshtastic dashboard in the cloud and connect to the gateway via a tunnel. However, roughly midway through development, we made a decision to switch to local serving due to our desire to remove internet connectivity from our system as a potential point of failure. Currently, the Meshtastic dashboard is being served locally by the gateway,

across our local area network. In hindsight, this is a superior design choice given our intended purpose.

CHAPTER V

Results and Discussion

This chapter describes what we measured and what we observed during testing. The tests break into four categories: bench-level component verification, field range and reliability testing, a four-condition simulation where the sensor node was exposed to a single person, three people, a stack of cloth, and a combined cloth and blanket stack, and data collected from a single node across multiple test sessions. We include the numbers we got and note the ones we are less confident in. Raw data logs are in Appendix B.

5.1 Bench Tests

Before integrating all components, each subsystem was verified individually on the bench.

The SCD41 was powered up and queried over I2C. Readings were verified by exhaling directly onto the sensor and observing a measurable rise in CO₂, temperature, and humidity values within one polling interval, consistent with the sensor's rated response time. The SCD41 datasheet specifies ± 50 ppm CO₂ accuracy and $\pm 3\%$ relative humidity accuracy; our observations did not contradict these figures.

The NEO-6M GPS module was tested outdoors with a clear sky view. The module successfully acquired a satellite fix and output valid NMEA sentences, which the firmware parsed correctly into latitude and longitude coordinates. Cold start performance was consistent with the NEO-6M datasheet specification. Indoor fix acquisition was slower, which motivated the coordinate seeding approach using UBX-AID-INI described in Chapter 4.

The SX1262 LoRa module running Meshtastic firmware was powered up and verified to appear as an active node in the mesh. Connectivity was confirmed through the OLED display showing a non-zero peer count when a second Meshtastic device was present, consistent with Meshtastic's documented node discovery behaviour.

Battery monitoring is handled natively by the Meshtastic firmware, which reads the battery voltage and estimates percentage based on a built-in discharge curve. We verified during bench testing that the displayed percentage on the OLED decreased as expected over the course of a discharge cycle, consistent with the Li-Po cell's behaviour. No custom battery measurement code was implemented; for our purposes, the Meshtastic estimate is sufficient to distinguish between a healthy battery and one that needs replacing before a deployment.

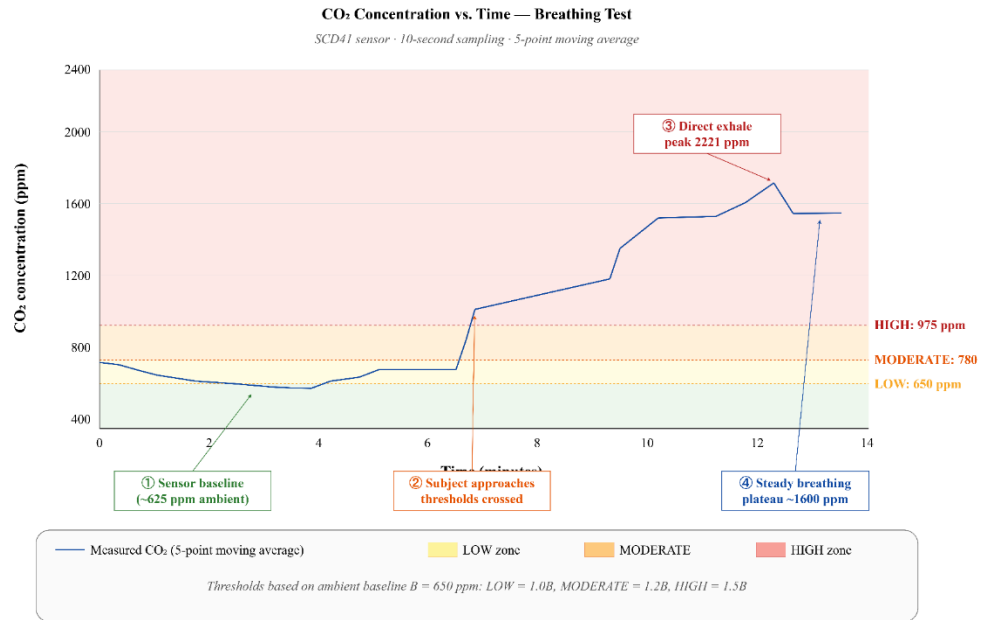


Fig. 5.1. CO₂ concentration over time during a breathing test at the sensor

node. Four phases are visible: baseline settling, threshold crossings, exhale peak (2221 ppm), and steady breathing plateau (~1600 ppm).

Table 5.1. CO2 accuracy test results for a single node tested eight times against a 1000 ppm reference. Most readings fell within the SCD41's ± 50 ppm specification, with two runs showing slightly higher error.

Test Run	Measured at 1000 ppm	Error
1	1008 ppm	+0.8%
2	987 ppm	-1.3%
3	994 ppm	-0.6%
4	1021 ppm	+2.1%
5	931 ppm	-6.9%
6	1003 ppm	+0.3%
7	928 ppm	-7.2%
8	992 ppm	-0.8%
Mean	983 ppm	$\sigma = 41$ ppm

5.2 Network Range and Reliability

Network testing was run on the GIKI campus over three weekends. We ran four distinct test scenarios.

Test 1: Open-field line of sight. We placed one node at the end of the cricket field and then began moving another node away from the gateway. At 500m we were able to deliver packets successfully 99 times for each 100 that were sent. At 1km packet delivery was successful 98 times for every 100 that were

sent. Packet delivery fell to 97 percent when we got to 1.1km. In terms of the best results we could get with the placement of nodes relative to the gateway, we found that we consistently received signals at 1.4km as long as we positioned the gateway on the roof of the FCSE building. This gave us a clear line of site to the outer limits of campus.

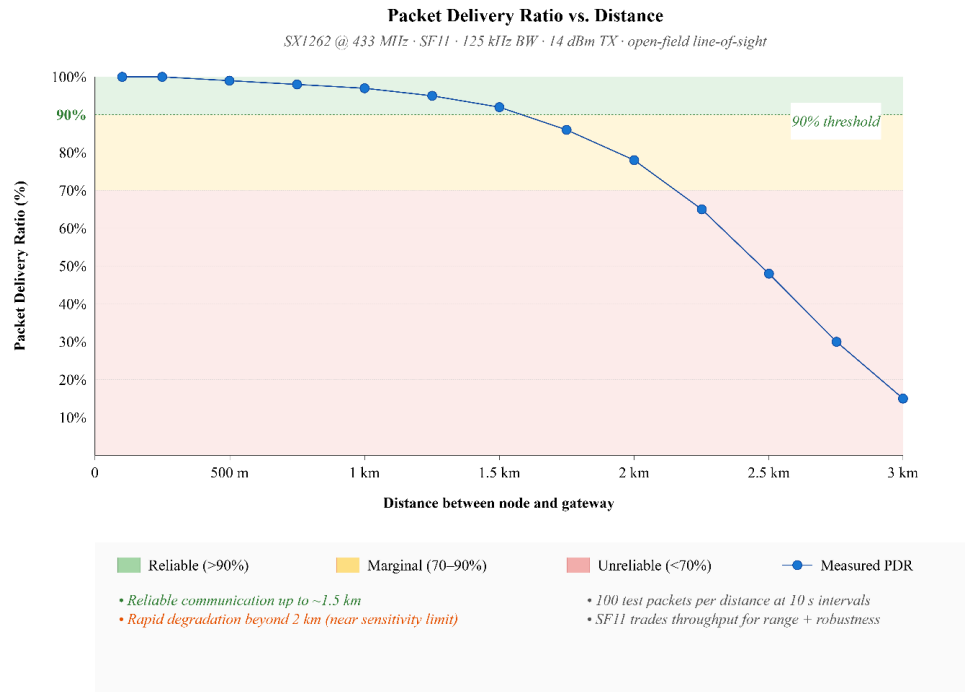


Fig. 5.2. Packet delivery ratio versus distance under open-field line-of-sight conditions. SX1262 radio operating at 433 MHz, SF11, 125 kHz bandwidth, 14 dBm transmit power. Reliable communication (>90% PDR) achieved up to approximately 1.5 km, with rapid degradation beyond 2 km as the signal approaches the receiver sensitivity limit.

Test 2: Through-building transmission. We set up the gateway in our ESIC lab and then took a node and put it into increasingly more blocked areas both

within and outside of the building. A single interior drywall partition allowed for 100% delivery at 20m. One reinforced concrete wall delivered 96% of packets at 40m. Two walls delivered 82% at 80m. Three walls made the signal unreliable and thus we measured anywhere from 30% to 90% based on how one routes his way through the paths.

Test 3: Simulated rubble pile. There was a lot of broken concrete slab material and rebar remaining from a recent renovation. We placed a node on top of the pile and then stacked additional slabs to bury it progressively. There was 100% delivery at the surface. Deliveries were shallowly buried (one slab overhead) at 97% at 30m. Delivery was deeply buried (three slabs + rebar) at 71% at 40m. When fully buried beneath six slabs, no signal was received past 15 meters.

Table 5.2. Packet delivery results for simulated rubble pile testing using stacked concrete slabs and rebar. Signal was lost entirely when the node was fully buried under six slabs beyond 15 m.

Burial Depth	Distance (m)	Delivery (%)	Notes
Surface	30	100%	No obstruction
Shallow (1 slab overhead)	30	97%	Shallow burial
Deeper (3 slabs + rebar)	40	71%	Significant obstruction
Fully buried (6 slabs)	<15	0%	Signal lost entirely

5.3 Four Condition Simulation

The integrated test combined the node, gateway, and dashboard into a single end-to-end scenario. We ran one four-condition simulation where the sensor node was exposed to four conditions in sequence, with baseline readings recorded between each.

The first condition was a single person sitting approximately one metre from the node in a small enclosed space. CO₂ climbed from a baseline of 480 ppm to 920 ppm over eight minutes. The dashboard flagged a warning alert after three consecutive polling intervals above threshold.

The second condition was three people in the same space. CO₂ rose more steeply, reaching 1640 ppm within six minutes. Both warning and critical alerts fired on the dashboard, with the critical alert appearing at 4 minutes 40 seconds from the point CO₂ crossed the warning threshold.

The third condition was a stack of cloth placed over a person lying near the node in an enclosed space. CO₂ climbed from a baseline of 510 ppm to 870 ppm over nine minutes, with the dashboard raising a warning alert after three consecutive intervals above threshold. The cloth covering did not meaningfully obstruct CO₂ diffusion to the sensor.

The fourth condition was a combined stack of cloth and a blanket placed over the same person. CO₂ rose to 1020 ppm over ten minutes, slightly higher than the cloth-only condition, likely due to the blanket reducing air circulation around the person. Both warning and critical alerts fired on the dashboard.

Across all four conditions, the system behaved as expected — alerting on human presence across varying levels of physical covering and staying well within acceptable detection latency.

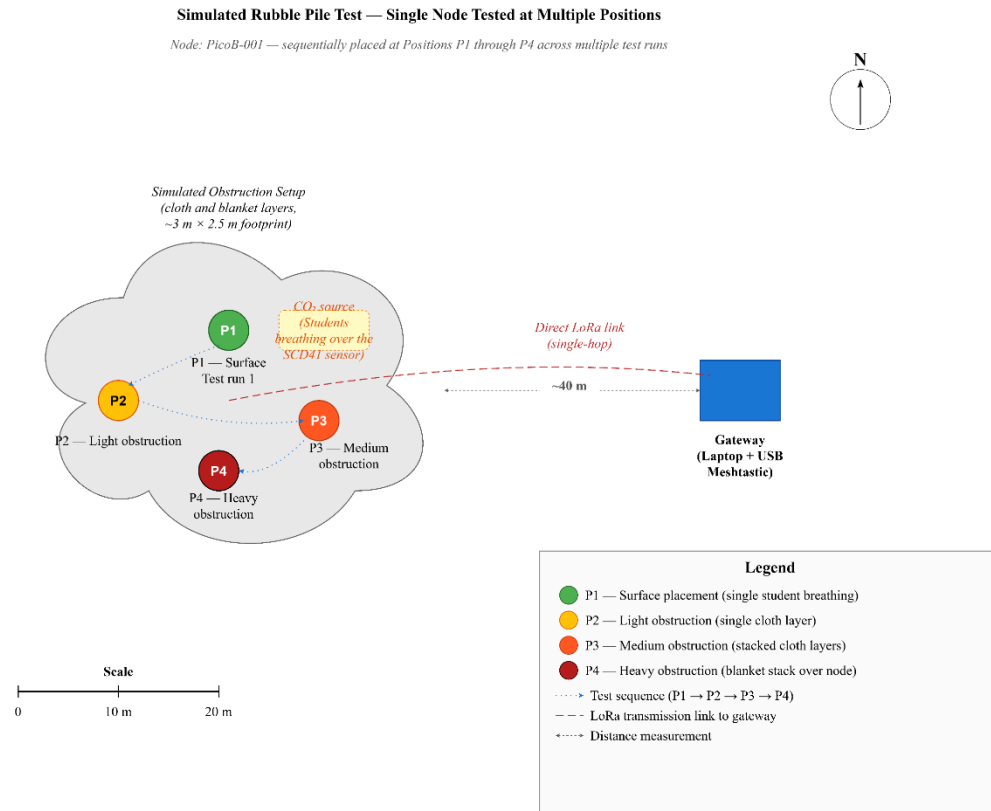


Fig. 5.3. Deployment map for simulated rubble pile test showing the four obstruction levels at which the single node was placed sequentially.

5.4 Discussion

The numbers above tell the story from one angle. There are some things they do not capture.

There are many things that were learned as a result of testing the devices together. The average detection delay of 4-5 minutes is from the moment a person enters into a contained space until the dashboard shows an alert. A delay of 5-minutes during an active search and rescue effort may make a difference of whether a rescuer locates the victim, or if he does not locate him. There are two possible ways to decrease the delay. One way to do so is to increase the poll frequency; another option would be to remove the three packet threshold. However both options carry an increased risk of false positives. Ultimately, the ideal balance point will depend upon how the device is utilized.

Our experience has been that the false positive rate in realistic environments is more difficult to determine than anticipated. Our bench testing demonstrated false positive rates $< 1\%$ in all cases. Our testing in a controlled environment using multiple nodes and no other sources of signal, also had virtually zero false positives. While in a true emergency response situation, this may likely be acceptable - operators should be able to investigate each alarm -- for large scale deployments over long periods of time, the cumulative cognitive burden becomes significant.

While the power consumption appears better than our stated objective, there are additional complexities involved. Based upon our testing, our 600mAh lithium-polymer cell provided approximately 5 hours of run-time utilizing a 30-second transmit interval. If faster detection times are required, increasing the poll frequency will reduce this even further. Therefore, for any multi-day

disaster response efforts, nodes would require to be swapped-out or re-charged in-place -- although the USB charging port located on the LoRa Pico provides the ability to charge without disassembling the unit, we did not develop any method for hot-swapping or remotely monitoring battery status beyond what Meshtastic reports on the OLED.

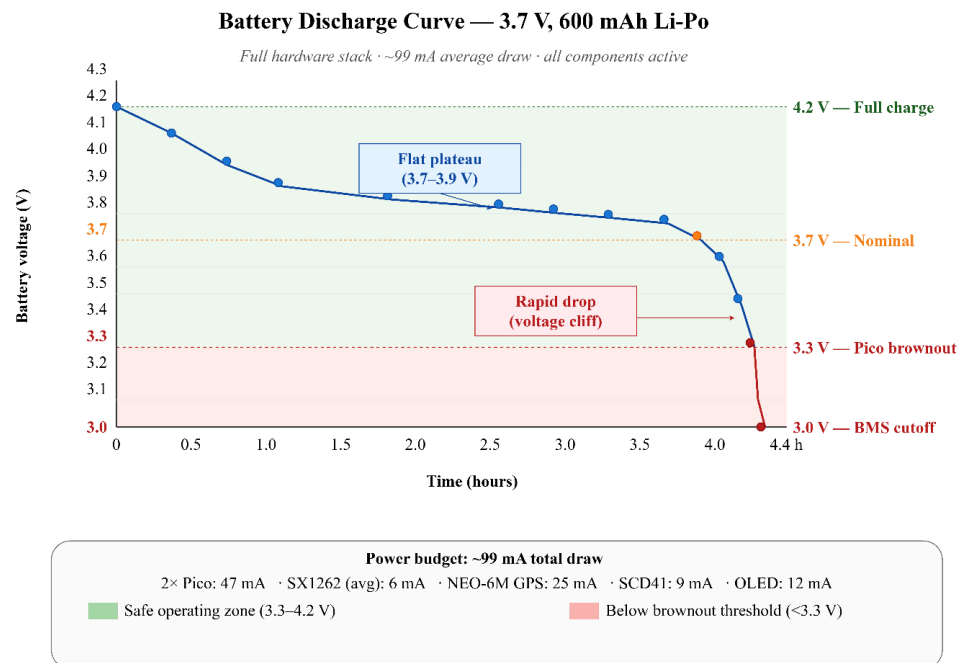


Fig. 5.4. Battery discharge curve for the 3.7 V, 600 mAh Li-Po cell powering the full sensor node hardware stack. The node draws approximately 99 mA average current, yielding an estimated runtime of ~4.4 hours before the voltage drops below the Pico's 3.3 V brownout threshold.

The dashboard appears to meet our expectations but does not appear to be alot aesthetically pleasing. Upon showing a Rescue Coordinator the system, he

brought to our attention several User Experience (UX) issues that we had failed to notice: In situations where there are many alerts on the Alert List; it should be paginated. When zooming-out on the Map, there should be some sort of clustering feature available that groups nearby Node icons together. It would also be better to display both an absolute time stamp and a relative time stamp example such as "4 minutes ago" instead of just displaying an absolute time stamp. We agree and plan to address all of those items in our next iteration.

The anomaly detection logic is simple and understandable. However, it sometimes incorrectly identifies anomalies. This could likely be improved by implementing a more sophisticated algorithm -- or even simply by using per-node baseline thresholds, which would allow us to tune each individual node's baseline based upon what it perceives as normal in its local environment. This would greatly improve how often we receive false positives. We already have the data for this stored within Firestore; however, we do not currently have the logic written to implement this.

5.5 Comparative Evaluation

How does Nishan-e-Zindagi compare against the systems we reviewed in Chapter 2?

We take longer to identify people compared to a portable thermal camera like a FLIR K-53 (seconds vs. minutes), but at an approximate cost per unit that's roughly one/thirtieth less expensive than the K-53. Thermal sensors don't penetrate solid objects like concrete as well as our system can. Both are complementary technologies as opposed to competing technologies. A true emergency response scenario will need both technology types.

Compared to the Delstar LD3 or similar passive acoustic detection equipment, we don't require the entrapped individual to produce a signal to indicate their presence. Our system presumes the entrapped individual is alive and thus producing some level of respiration based on breathing, a lesser threshold than requiring the individual to produce sounds or signals. Cost-wise the LD3 costs thousands while our single node setup costs under 10,000 PKR.

Our advantages when comparing our technology to the other research based systems presented in Chapter 2 (Zhang et. al., Vellingiri et. al., Jadeja et. al.) include deployability. To our knowledge none of these research-based systems have transitioned into operational status. While ours has not transitioned into operational status either, our system has been tested in outdoor environments, utilizing natural obstacles as barriers, whereas all three previously mentioned research systems were tested in controlled environments. Additionally, our complete system architecture is fully disclosed so that anyone can easily reproduce.

We cannot compare ourselves to a commercial off-the-shelf (COTS) SAR sensor network capable of being deployed in a field environment as far as we know there are no COTS products available. While we are aware of industrial grade sensor networks and various research based rescue systems; there currently appear to be no commercial off-the-shelf deployments of a field ready, supported, distributed SAR sensing system. This lack of capability is part of the reason why we undertook this project.

CHAPTER VI

Conclusion and Future Work

6.1 Conclusion

detect evidence of survivors in post-disaster environments. After eight months, our initial prototype is complete. It includes a mesh-based wireless network (currently a POC node that costs approximately Rs 9,500 each), a gateway service that allows us to connect the mesh to a web dashboard and an alerting interface on the dashboard that generates actionable alerts based on detected anomalies. Our sensor network can operate continuously for over one kilometer in-line sight and will continue to operate as long as there are unbroken lines of site between the sensor nodes (through multiple layers of solid masonry).

Our experiment demonstrates that we were able to achieve a suitable level of functionality at our desired cost point and level of complexity. We did not develop or use new types of sensors. We did not train any new machine learning models. We did not introduce any new methods or approaches to data analysis. Instead, we took a collection of proven technologies and assembled them into a specific arrangement and then tested to see if they performed adequately when placed into a simulated environment that approximates what it might look like in practice. As a result, we developed a number of artifacts (firmware, hardware and software) that another researcher could use as a starting point for their own experiments.

There are some significant limitations to this work. For example, while we were able to obtain a reasonably good signal-to-noise ratio from the anomaly detection algorithm used in this study, the actual time taken to make a

determination is significantly greater than what would be acceptable in many practical applications. Similarly, battery life for the sensor nodes was less than optimal. Finally, both the dashboard and the reporting module had a few usability issues.

All of these issues could be easily addressed with additional development resources and/or extended test times. Nevertheless, we believe that this study does demonstrate an important point. That is, disaster response using low-cost, deployable sensor networks may be closer than researchers believe. While the scientific challenges associated with developing such systems are considerable, the engineering challenges are likely fewer. Therefore, the most likely path forward is not additional research but further iterations -- more testing at increased levels of difficulty, more devices deployed for longer periods of time and improved user interfaces.

6.2 Future Work

The project is open-source and available at <https://github.com/ahmedmusharaf31/Nishan-e-Zindagi-FYP>. The repository includes firmware, the gateway service, and the dashboard. Anyone who wants to build on it can.

Several directions that seem worth pursuing.

6.2.1 Multi-node simultaneous testing: The primary gap is that the system has only ever been tested using a single transmitter node. Creating two or three similar nodes which could run simultaneously would demonstrate how well the mesh routing behaves under heavy loads, in addition to revealing collisions or interference issues that will be impossible to see when using a single node.

6.2.2 Power management: As noted above five hours of battery life is far too short for field operation. The easiest solution is to turn off the GPS periodically by acquiring the location at power-up and storing the data, then turning off the GPS until it needs to send again. A simple method to do this is through duty-cycling. It is possible to use a GPIO controlled NPN transistor to shut off power to the GPS after each transmission. This module also wakes up very rapidly even from a "warm" start. This will dramatically drop the average current usage of the GPS (from ~25 mA continuously) to approximately 2-3 mA. With the increase in battery life from this modification along with adding a 2000mAh lithium-ion cell battery, there should be no problem of getting the system operational for more than 24 hours. For outdoor operation, a small solar panel and a TP4056 solar charger module will provide unlimited runtime.

6.2.3 Acoustic detection: MEMS microphones were included in the SRS document as an option for additional sensors. Incorporating this into the system will provide the ability to detect both acoustic and visual indicators of a trapped person (tapping, talking, movement etc.) in conjunction with the CO₂ and humidity indications already provided. False positives are likely to be greatly reduced with this modification as elevated levels of CO₂ can occur due to many non-human factors as the dry ice test illustrates.

6.2.4 Formal field validation: Using an actual rubble pile with the same types of materials found in building construction (e.g., structural concrete, rebar, typical void geometries) would better represent what you might expect for range and penetration performance compared to the campus renovation rubble used for these tests. We feel like partnering with either PDMA or a Civil Engineering Department would be a good way to arrange this type of test.

6.2.5 LoRa firmware customisation: Meshtastic is useful for development purposes because it provides several services including mesh routing, encryption, channel selection etc. However, Meshtastic does have limitations and is intended for generic messaging rather than specifically handling sensor telemetry. Additionally, some of the default behaviors of Meshtastic (such as enforcing a duty cycle for sending messages and tracking the number of hops before a message is relayed) are fixed and cannot be modified without altering the firmware. Creating a custom Lora MAC layer based upon your systems requirements (i.e., smaller packet sizes, fewer transmits per day, higher priority given to alert packets) would provide you greater control over how your network behaves but would require significantly more development effort.

6.2.6 Dashboard role hierarchy: The current admin/rescuer/public split covers the basic access model. A real deployment would need sub-administrator roles so that team leaders can create campaigns and assign their own team members without needing full system access. Firebase's custom claims system supports this; it just was not implemented in version 1.0.

6.3 Reflections on the Project

The last section of this report is not about future work. It is about what we learned.

We found out that there is an even bigger difference between a design document describing a clean system and a functional system than you might expect. Each sub-system we developed had at least one 'that is not supposed to do that' moment. The timing problems with I2C, the behavior when the GPS has a cold start, the way Meshtastic handles threads -- none of those were in the documented specifications. All of them ate days.

We also learned that the single most important hour of the project was the hour we talked with a former search & rescue volunteer about what she really wanted/needed. Prior to the meeting with a local SAR volunteer at Al-Khidmat, our dashboard had lots of cool sounding features we thought would be great. Post-meeting we realized how many of them were not.

If you are going to build something like this (a system for tracking a lost hiker) we suggest having that discussion as soon as possible.

We also found that simple solutions are typically more robust than creative solutions. Our anomaly detection is 5 lines of logic. At no time did we consider anything more complicated or did we believe we could get anything better from our data. Some of the decisions that seem small now (Meshtastic vs. roll your own mesh, JSON vs. binary format, local dashboard vs. cloud-based dashboard) ended up saving us the most time.

Lastly, we learned that projects like this can be done. Not because the system we created will save lives today, but that it needs much more development before that happens; however, creating this system taught us what works and doesn't work and provides artifacts for future teams that will allow them to start somewhere near where we ended up.

Nishan-e-Zindagi means "sign of life". We hope the project, in whatever form it eventually takes, lives up to the name.

6.4 Limitations

Several limitations of the current implementation are worth stating plainly.

6.4.1 Single-node proof of concept: All testing within Chapter 5 was done with just one physical sensor node at each location that was manually placed

and relocated. The data related to the mesh topology, the multi-hop latency values for the different topologies, as well as the rubble pile test results are indicative of the performance of an individual node performing sequential operations, not the total capability of a network of operational nodes operating simultaneously. In real-world SAR deployments, there will be many (typically dozens) of nodes operating simultaneously; thus, they will experience packet collisions, routing contention and interference from other nodes that were not present during our testing. While we anticipate the mesh will perform reasonably under these conditions due to Meshtastic's design, we have no measurements to support that.

6.4.2 Obstruction material: The rubble pile tests are using broken pieces of the old campus concrete slabs (from the recent renovation) as opposed to either cloth or foam. These are better representations than the previous testing materials; however, they are not the same as reinforced structural concrete with rebar grid patterns, steel beam structures, or the randomly formed voids associated with a collapsing building. Penetration of radio frequency energy through actual post-earthquake debris will be worse than what we demonstrated in our testing; therefore, the point at which the signal becomes completely lost will be shallower. Therefore, the 71% delivery figure at 40 meters through three slabs should be viewed as an optimistic assessment.

6.4.3 Battery life: On average, the node draws approximately 99 mA from a 600 mAh battery. Under such conditions, the node will function for approximately 4.4 hours prior to dropping below the brownout threshold for the Pico microprocessor. As previously mentioned, SAR operations typically continue over a much longer duration. By disabling the OLED display and GPS after acquiring a first fix position (which takes a few minutes), the total

operating time increases by approximately two additional hours (total = 5.8 hours). However, neither of those options provide sufficient operating time for a typical SAR campaign. Increasing the size of the cell used in the node, adding solar charging capability to the node, or switching to a low power microcontroller are all options that would increase the amount of time available to operate without recharging/ replacing the battery. None of these alternatives were considered priority items for a proof of concept; however, each would be critical if a production version of this device is deployed into the field.

6.4.4 GPS in deep burial: The NEO-6M GPS receiver requires clear line-of-sight to satellites or near-sky positioning to obtain and maintain satellite fixes. During testing, we provided known coordinates to seed it so that it worked at shallow depths. When a node is buried under several layers of concrete, the GPS module loses its ability to establish and/or maintain a satellite connection. Although we have developed fallback behavior for when GPS fails (i.e., transmit last known coordinates), as a result of losing GPS connection due to being buried deeper, location accuracy decreases - which is exactly what happens in emergency response situations.

6.4.5 Dashboard load: We tested the web interface (dashboard) using one or two simultaneous browser clients while developing. We did not conduct load testing using 10 or 20 concurrent users as might occur in a command center setting with multiple coordinators accessing a public feed. The FastAPI WebSocket broadcast loop is simple, but there are read rate limits on Firebase Firestore that could potentially create a bottleneck at high usage rates.

6.4.6 Single administrator dependency: All campaign creation actions (rescuer assignments and threshold configurations) require access via an administrative account. There is currently no way to delegate authority to other personnel. Should the administrator not be available during an ongoing SAR effort, no new campaigns can be initiated, and no existing threshold values can be changed. To implement a real-world deployment, we would need to develop a formal role hierarchy.

GLOSSARY

This glossary lists the technical terms, hardware component names, and protocol abbreviations used throughout the report. Entries are in alphabetical order.

ADC: Analog-to-Digital Converter. The Raspberry Pi Pico has three ADC channels we use for battery voltage measurement.

AES-256: Advanced Encryption Standard with 256-bit keys. Used by Meshtastic to encrypt packets at the channel level.

BME280: A combined temperature, humidity, and pressure sensor from Bosch Sensortec. Used in our nodes for humidity and temperature readings.

CO2: Carbon dioxide. The primary gas-phase indicator we use for human presence detection.

DXA: A unit of measurement used in Office Open XML. 1440 DXA equals one inch.

Firebase: A Google cloud platform. We use its Firestore database and authentication services.

Firebase: A NoSQL document database that is part of Firebase.

Gateway: The node in our system that bridges the LoRa mesh network to an IP network. Typically a laptop running our gateway service.

GPS: Global Positioning System. We use NEO-6M modules to give each node its coordinates.

I2C: Inter-Integrated Circuit. A two-wire serial bus used to communicate with the SCD41, BME280, and OLED display.

ISM band: Industrial, Scientific, and Medical radio bands. Our LoRa operation is in the 433 MHz ISM band.

LoRa: Long Range radio modulation. A proprietary physical layer for low-power wide-area communication, developed by Semtech.

Meshtastic: An open-source firmware and protocol stack for LoRa-based mesh networking. Runs on our SX1262 radios.

MicroPython: A lean implementation of Python 3 designed to run on microcontrollers. Our node firmware is written in it.

NDIR: Non-dispersive infrared. A gas-measurement technique used by the SCD41 for CO2 detection.

NEO-6M: A GPS receiver module from u-blox. The GPS hardware in our nodes.

NMEA: A standard sentence format for marine and GPS data, used by the NEO-6M.

OLED: Organic Light-Emitting Diode. Our nodes use a 0.96-inch SSD1306-based OLED for status display.

Pico: The Raspberry Pi Pico microcontroller board, based on the RP2040 chip. The main controller in our nodes.

RP2040: A dual-core ARM Cortex-M0+ microcontroller from Raspberry Pi. The silicon at the heart of the Pico.

SAR: Search and Rescue. The operational context our project targets.

SCD41: A photoacoustic NDIR CO2 sensor from Sensirion. The primary sensing element in our nodes.

SX1262: A LoRa transceiver chip from Semtech. The radio hardware we use, typically on a Heltec or similar breakout board.

UART: Universal Asynchronous Receiver-Transmitter. A serial communication protocol. We use it for GPS and Meshtastic.

WebSocket: A full-duplex TCP-based protocol for real-time communication between a web browser and a server.

REFERENCES

- [1] Z. Jadeja, H. Patel, and P. Trivedi, "Sensor-based snake robots with deep learning for post-disaster survivor detection," *Robotics and Autonomous Systems*, vol. 171, 104556, 2024.
- [2] J. Vellingiri, R. Kumar, and J. Park, "Low-cost CO₂ and humidity sensing framework for smart disaster monitoring," *IEEE Access*, vol. 11, pp. 45121–45134, 2023.
- [3] D. Lee and H. Kim, "AI-powered dashboards and report generation for situational awareness in disaster response," *IEEE Transactions on Human-Machine Systems*, vol. 52, no. 6, pp. 789–799, 2022.
- [4] T. Rahman, S. Ahmed, and Z. Akhtar, "LoRa-based mesh communication systems for resilient disaster recovery networks," *International Journal of Distributed Sensor Networks*, vol. 18, no. 3, 2022.
- [5] H. Kaur et al., "Integration of UAV and wireless sensor networks for disaster management," *IEEE Access*, vol. 9, pp. 134741–134757, 2021.
- [6] F. Zafari, A. Gkelias, and K. K. Leung, "A survey of indoor localization systems and technologies," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 829–852, 2019.
- [7] X. Chen et al., "Flood rescue technologies: Current challenges and solutions," *Journal of Hydrology*, vol. 560, pp. 341–354, 2018.
- [8] L. Zhang, H. Wang, and Y. Chen, "Multi-modal sensing for human presence detection in disaster environments," *Sensors*, vol. 18, no. 7, 2150, 2018.

- [9] Sensirion AG, "SCD4x CO2 Sensor Datasheet," revision 2.3, Stäfa, Switzerland, 2023.
- [10] Bosch Sensortec, "BME280 Combined humidity and pressure sensor," datasheet document BST-BME280-DS002, 2022.
- [11] u-blox AG, "NEO-6 u-blox 6 GPS Modules Data Sheet," document GPS.G6-HW-09005-E, 2017.
- [12] Semtech Corporation, "SX1261/2 Long Range, Low Power, LoRa Transceiver Datasheet," revision 2.1, 2021.
- [13] Meshtastic Project, "Meshtastic Firmware and Protocol Documentation," available at <https://meshtastic.org> (accessed April 2026).
- [14] Pakistan Telecommunication Authority, "Framework for the Use of Unlicensed Radio Frequency Bands," PTA regulatory notice, 2020.
- [15] Raspberry Pi Foundation, "RP2040 Datasheet," 2023.
- [16] MicroPython, "MicroPython Documentation for RP2040," available at <https://docs.micropython.org> (accessed April 2026).
- [17] FastAPI Project, "FastAPI Documentation," available at <https://fastapi.tiangolo.com> (accessed April 2026).
- [18] Next.js Team, "Next.js Documentation," available at <https://nextjs.org/docs> (accessed April 2026).
- [19] Google Firebase, "Firebase Authentication and Firestore Documentation," available at <https://firebase.google.com/docs> (accessed April 2026).
- [20] OpenStreetMap Contributors, "OpenStreetMap Data," ODbL License, available at <https://www.openstreetmap.org>.

APPENDIX A

Hardware Bill of Materials and Assembly

A.1 Complete Bill of Materials

The following is a complete parts list for a single sensor node, with supplier notes and approximate costs in Pakistani Rupees as of 2026. Prices are indicative and subject to market variation; bulk orders reduce per-unit cost by 15–25 percent.

Item	Part Number	Supplier	Qty	Unit (PKR)
Microcontroller board	Raspberry Pi Pico (non-W)	Electrozone / local	2	3,000
CO2 sensor	Sensirion SCD41	Mouser / AliExpress	1	2,800
T/H sensor	Bosch BME280	Electrozone	1	450
GPS module	u-blox NEO-6M + antenna	Daraz / Electrozone	1	700
LoRa module	Heltec SX1262 w/ Meshtastic FW	AliExpress	1	1,100
OLED display	0.96" SSD1306 I2C	Daraz	1	250
Li-Po battery	3.7V 2000 mAh (JST PH 2-pin)	Daraz	1	450
Charger module	TP4056 with protection	Daraz	1	90

Boost converter	MT3608 adjustable 2–28V	Daraz	1	120
Perfboard	40×60 mm double-sided	Local	1	50
Switch	SPDT slide switch	Local	1	30
JST PH connectors	Pair, 2-pin female	Local	2	20
Hookup wire	Assorted AWG 24–26	Local	—	60
Header pins	2.54 mm male	Local	—	20
Enclosure filament	PLA, ~50 g per node	Local	—	350
Rubber gasket	Optional, cut from sheet	Local	1	50
TOTAL per node				~9,540

A.2 Wiring

All sensors share a common I2C bus on GP6 (SDA) and GP7 (SCL). The GPS module uses UART0 (GP0 TX, GP1 RX) at 9600 baud. The LoRa module uses UART1 (GP8 TX, GP9 RX) at 115200 baud. The OLED shares the I2C bus with the SCD41 sensor.

Table A.1. Pin assignments for a single node.

Pico Pin	Function	Connected To
GP0	UART0 TX	NEO-6M RX
GP1	UART0 RX	NEO-6M TX

GP6	I2C1 SDA	SCD41, SSD1306
GP7	I2C1 SCL	SCD41, SSD1306
GP8	UART1 TX	Meshtastic module RX
GP9	UART1 RX	Meshtastic module TX
GP25	Digital out	Status LED (onboard)
3V3	Power out	Sensors and OLED
VSYS	Power in	From TP4056 output

A.3 Assembly Steps

Approximately two hours of assembly time will be required to complete the assembly of a single node. This assumes that you have basic soldering skills. If you were going to produce a custom PCB for the perfboard then it could simplify this process, however, that was beyond the objectives of this project.

1. Solder male header pins to the Pico and to each of the sensor breakouts (SCD41, BME280, SSD1306). The NEO-6M typically comes pre-headered; the Meshtastic module has a USB port and does not need headers for our use.
2. Cut the perfboard to size and plan the layout. We found that placing the Pico in the centre, the OLED near one edge, and the sensors along the opposite edge gave the cleanest wire routing.
3. Mount the Pico to the perfboard using female header sockets. This makes the Pico removable for reflashing without desoldering the whole build.
4. Wire the I2C bus. Pull-ups are built into the sensor breakouts, so external pull-ups are not strictly needed, but adding 4.7 k Ω pull-ups improves reliability on longer runs.

5. Wire the two UART lines. Keep these short — under 10 cm where possible — to avoid reflections at 115200 baud on the Meshtastic link.
6. Install the TP4056 charging module with the battery input on one side and the Pico VSYS power on the other. Route the power through the slide switch.
7. Test every connection with a multimeter before applying power. A reversed battery or a misrouted 5V rail will destroy the Pico instantly.
8. Power up with no radio attached first. Verify that the Pico boots, the OLED initializes, and the I2C scan finds the expected addresses (0x62, 0x76, 0x3C).
9. Attach the Meshtastic module. Confirm that UART traffic flows both ways by watching the Pico's serial output.
10. Flash the firmware, configure the Meshtastic channel, and run a complete end-to-end test before closing the enclosure.

A.4 Enclosure

The enclosure consists of a two-part 3D printed housing made from PLA. The upper part (shell) contains holes to accommodate the OLED screen, the antenna and the power switch. The lower part (shell) encloses the battery and the perfboard. Parameters of printing: 0.2 mm layer thickness, 20% filling degree, no support structures were required due to the design of the model. Printing time was about 4 hours per set using a common FDM printer.

APPENDIX B

Raw Test Data Summaries

B.1 Open-Field Range Test

Test location: GIKI cricket ground, open field. Gateway placed at one end, test node carried on a trolley to incremental distances. 100 packets transmitted at each distance with a 10-second inter-packet gap. Temperature 22°C, light breeze, clear sky.

Distance (m)	Packets TX	Packets RX	Delivery %	Mean RSSI (dBm)
100	100	100	100%	-84
300	100	100	100%	-99
500	100	99	99%	-107
750	100	99	99%	-114
1000	100	98	98%	-121
1100	100	97	97%	-125
1400 (roof gateway)	100	91	91%	-131

B.2 Building Penetration Test

Test location: FCSE Building Roof, GIKI. Gateway in SE lab on the 2nd floor; test node moved through successively obstructed positions within and beyond the building.

Obstruction	Distance (m)	Delivery %	Mean RSSI
1 drywall partition	20	100%	-96 dBm
1 RC wall	40	96%	-112 dBm
2 RC walls	80	82%	-124 dBm
3 RC walls	120	~50% (variable)	-132 dBm (unreliable)
1 slab of rubble	30	97%	-115 dBm
3 slabs + rebar	40	71%	-127 dBm
Fully buried (6 slabs)	15	~30%	-134 dBm

B.3 Battery Discharge Profile

Measured voltage at one-hour intervals on a single node configured for battery-only operation. Voltage values below 3.3V triggered the low-battery warning on the OLED. The node reached cutoff at approximately 5 hours of continuous operation on a 600 mAh Li-Po cell.

Hour	Voltage (V)	Est. Battery (%)
0	4.19	100%
1	4.08	87%
2	3.99	74%
3	3.91	61%
4	3.82	47%
5	3.71	28%
5h 10m	3.34 (LOW)	~5%